PLATAFORMA DE VOTACIÓN ELECTRÓNICA

YESID DAVID JURADO CONTRERAS

UNIVERSITARIA TECNOLOGICA DE SANTANDER
FACULTAD DE CIENCIAS NATURALES E INGENIERIAS
INGENIERIA DE SISTEMAS
BUCARAMANGA
2026

Contenido

Contextualización del Tema 3
Objetivos del Proyecto 4
Manual de uso de la plataforma electoral 5
Patrón de Diseño Singleton 11
Patrón de Diseño Factory Method 14
Patrón de Diseño Abstract Factory 17
Patrón de Diseño Builder 21
Patrón de Diseño Prototype 26
Patrones de Diseño Adapter y Bridge 29
Patrones de Diseño Composite, Decorator y Facade 38
Patrones de Diseño Proxy y Flyweight 44

Contextualización del Tema

1.1 ¿Qué es una Plataforma de Votación Electrónica?

Una plataforma de votación electrónica es un sistema de información diseñado para registrar, procesar, tabular y auditar sufragios de manera digital, ya sea en puestos de votación presenciales (mediante dispositivos de registro directo o DRE) o de forma remota a través de redes seguras.

A nivel global, la adopción del voto electrónico busca mejorar la eficiencia en la tabulación de resultados, reducir el uso de papel, eliminar errores humanos en el escrutinio de mesas y permitir la entrega de resultados preliminares en tiempos reducidos. Países como Estonia han consolidado modelos de voto por internet, mientras que otros como Brasil utilizan urnas electrónicas presenciales con respaldo digital.

1.2 Problemática de los Procesos Electorales Tradicionales

Los métodos de votación tradicionales basados en papeletas físicas presentan desafíos recurrentes que impactan la eficiencia y la confianza en los resultados:

- **Errores humanos y lentitud en el escrutinio:** El conteo manual de votos es un proceso dispendioso y propenso a equivocaciones al momento de tabular y diligenciar las actas físicas.
- **Riesgos de custodia e integridad:** El transporte y almacenamiento de material electoral físico implica riesgos de extravío, daño o alteración de votos.
- **Altos costos logísticos:** Requieren una gran inversión de recursos en la impresión de tarjetones, distribución de material y despliegue de personal.
- **Dificultad de auditoría y verificación:** El recuento físico de votos ante controversias o inconsistencias es complejo y demorado, lo que suele generar incertidumbre en los participantes.

1.3 El Dilema Central del Voto Digital

En la ingeniería de software, diseñar un sistema electoral representa un reto singular debido a que debe satisfacer dos principios fundamentales:

1. **Secreto del Voto (Anonimato):** Nadie (ni los administradores del sistema, ni los auditores, ni otros votantes) debe poder asociar la identidad de un ciudadano con su elección específica.
2. **Integridad y Verificabilidad (Auditabilidad):** Cada voto emitido debe ser registrado exactamente como el votante lo expresó, sin alteraciones, y debe ser posible auditar que el conteo final corresponde con la totalidad de los votos emitidos.

Adicionalmente, se deben mitigar riesgos como la coerción, el fraude informático, la suplantación de identidad, la denegación de servicio (DoS) y la pérdida de confianza de los participantes.

Objetivos del Proyecto

2.1 Objetivo General

Desarrollar un prototipo funcional de una **Plataforma de Votación Electrónica** que permita simular un proceso electoral, facilitando la autenticación de votantes, la emisión anónima del voto, el conteo automatizado de resultados y el registro de auditoría de la jornada.

2.2 Objetivos Específicos

1. **Permitir la configuración de la jornada electoral:** Facilitar el registro de los candidatos y opciones de votación necesarios para llevar a cabo el proceso electoral.
2. **Implementar el control de acceso y validación de votantes:** Desarrollar un mecanismo para autenticar a los electores y verificar que solo puedan ejercer su derecho al voto una única vez.
3. **Garantizar el secreto y la privacidad del voto:** Diseñar la lógica de emisión de papeletas digitales de forma que la identidad del usuario no quede vinculada a la opción seleccionada.
4. **Construir el motor de conteo automatizado:** Desarrollar el módulo encargado de registrar, tabular y totalizar los votos emitidos para presentar los resultados del escrutinio de manera precisa.
5. **Crear un registro de eventos y auditoría:** Implementar un sistema de registro cronológico (*logs*) para rastrear eventos clave (apertura de votación, emisión de votos, cierre de jornada y emisión de resultados), permitiendo verificar la transparencia del proceso.

Manual de uso de la plataforma electoral

1. Antes de empezar

Asegúrate de tener Python 3.12 o superior. Activa el entorno virtual e instala las dependencias:

```
source venv/bin/activate
pip install -r requirements.txt
```

1.1 Arranque

Desde la raíz del proyecto:

```
python3 -m src.main
```

La aplicación queda disponible en los siguientes módulos:

Módulo	URL
Cabina de votación	http://127.0.0.1:8000/
Resultados en vivo	http://127.0.0.1:8000/dashboard
Acceso administrativo	http://127.0.0.1:8000/admin/login
Documentación de la API	http://127.0.0.1:8000/docs

1.2 Credenciales de demostración

Con estas credenciales sintéticas puedes iniciar sesión en la cabina de votación o en el acceso administrativo.

- **Administrador:** usuario `admin` y contraseña definida en `.env` (por defecto `admin1234`).
- **Electores:** todos usan el PIN `1234`.
- **Documentos nacionales:** `10203040`, `50607080`, `70809010`, `80901020`, `20000002`, etc.
- **Documentos universitarios:** `EST-2024`, `EST-9012`, `EST-3341`, `EST-5520`, `EST-1002`, etc.

Modalidad	Electores	Habilitados	Inhabilitados
Nacional (CNE)	41	37	`99999999`, `20000001`, `20000013`, `20000025`
Universitaria (UDI)	41	37	`EST-9999`, `EST-1001`, `EST-1013`, `EST-1025`

El PIN `1234` es una credencial de simulación. No representa un mecanismo de autenticación de nivel electoral.

2. Recorrido del elector

2.1 Identificación

El elector abre la cabina, escribe su documento dependiendo de la elección que esté habilitada en ese momento, sea UDI o CNE, junto con su respectivo PIN, y pulsa **Verificar elector**.



Figura 1. Cabina de votación: identificación del elector.

El sistema valida:

1. Que la jornada esté abierta.
2. Que el formato del documento corresponda a la modalidad.
3. Que el documento exista en el censo y esté habilitado.
4. Que el PIN sea correcto.
5. Que el elector no haya votado ya en esa elección.

Si algo falla, se muestra un mensaje sin revelar si el documento existe o no.

2.2 Selección del tarjetón

Una vez verificado, el elector ve el tarjetón oficial con los candidatos y la opción de **Voto en Blanco**.

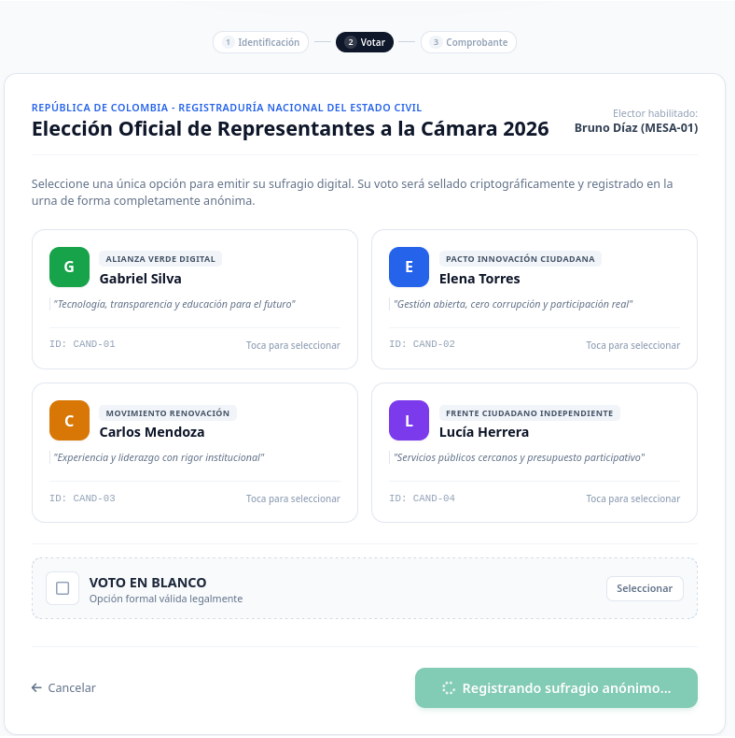


Figura 2. Selección del tarjetón y voto en blanco.

- Puede seleccionar un candidato o el voto en blanco.
- El botón **Confirmar y emitir sufragio** se habilita solo cuando hay una selección.
- El servidor recibe únicamente el identificador de la opción; el nombre y el partido los resuelve el sistema.

2.3 Comprobante

Al confirmar, se muestra un comprobante con:



Figura 3. Comprobante digital del voto.

- Código de verificación.
- ID de papeleta anónima.
- Sello de tiempo UTC.
- Firma HMAC-SHA256 (integridad).

El comprobante no contiene el documento del votante.

2.4 Resultados públicos

El enlace **Ver resultados en vivo** lleva al dashboard:

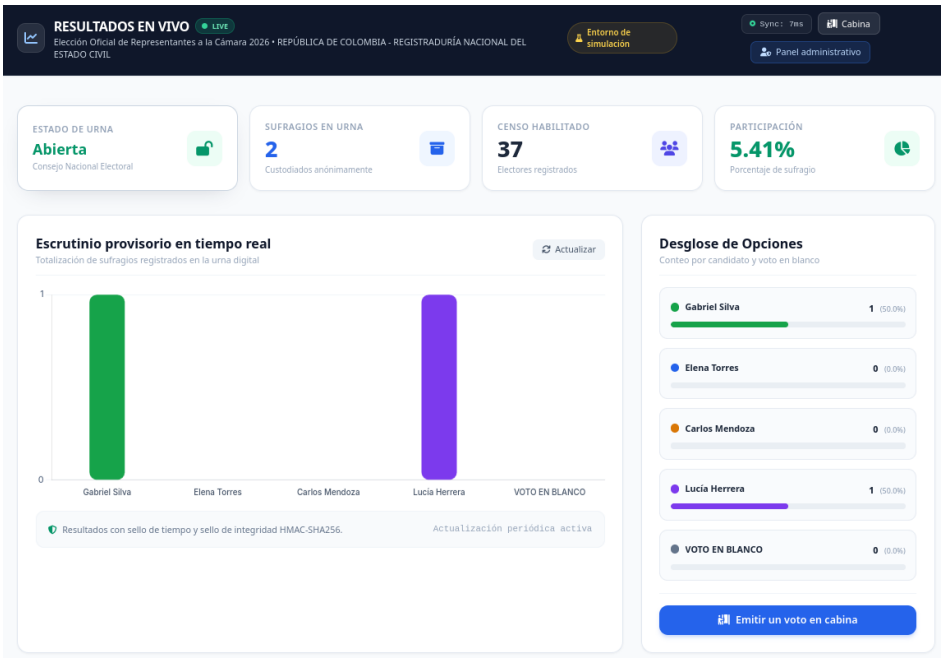


Figura 4. Panel público de resultados en vivo.

- Estado de la urna.
- Total de sufragios.
- Censo habilitado.
- Porcentaje de participación.
- Desglose por candidato y voto en blanco.

El dashboard se actualiza cada 5 segundos.

3. Panel administrativo

3.1 Acceso

La ruta `/admin` redirige a `/admin/login` si no hay sesión.

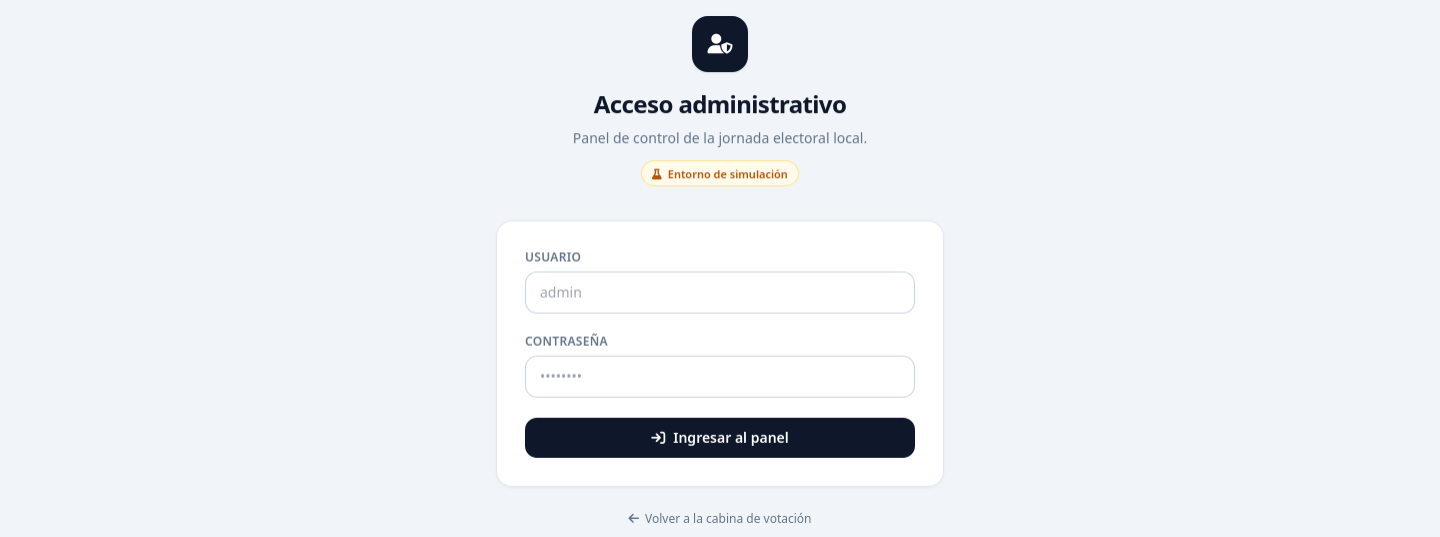


Figura 5. Acceso al panel administrativo.

El acceso usa una única cuenta local. La sesión se guarda en una cookie `HttpOnly`; el JavaScript del panel no ve el valor de la sesión.

3.2 Funciones del panel

Función	Qué hace
Cambiar modalidad	Alterna entre Elección Nacional y Universitaria. Los votos de cada modalidad se conservan por separado.
Control de urna	Abre o cierra la jornada. Cerrar bloquea nuevos votos; si no se generó acta, se puede volver a abrir.
Cierre y acta	Cierra la urna, cuenta los votos, calcula el ganador y aplica el sello SHA-256. No se puede deshacer.
Reiniciar jornada	Borra los votos y participaciones de la elección activa; **conserva el censo**.
Censo electoral	Lista, inscribe, actualiza y habilita/inhabilita electores.
Bitácora	Muestra los eventos de verificación, emisión, apertura, cierre, acta y reset.

El acta es la finalización de la jornada: después de generarla, la urna queda cerrada y escrutada. Para repetir, usa **Reiniciar jornada**.

No existe un borrado de voto individual: eso obligaría a vincular identidad y papeleta.

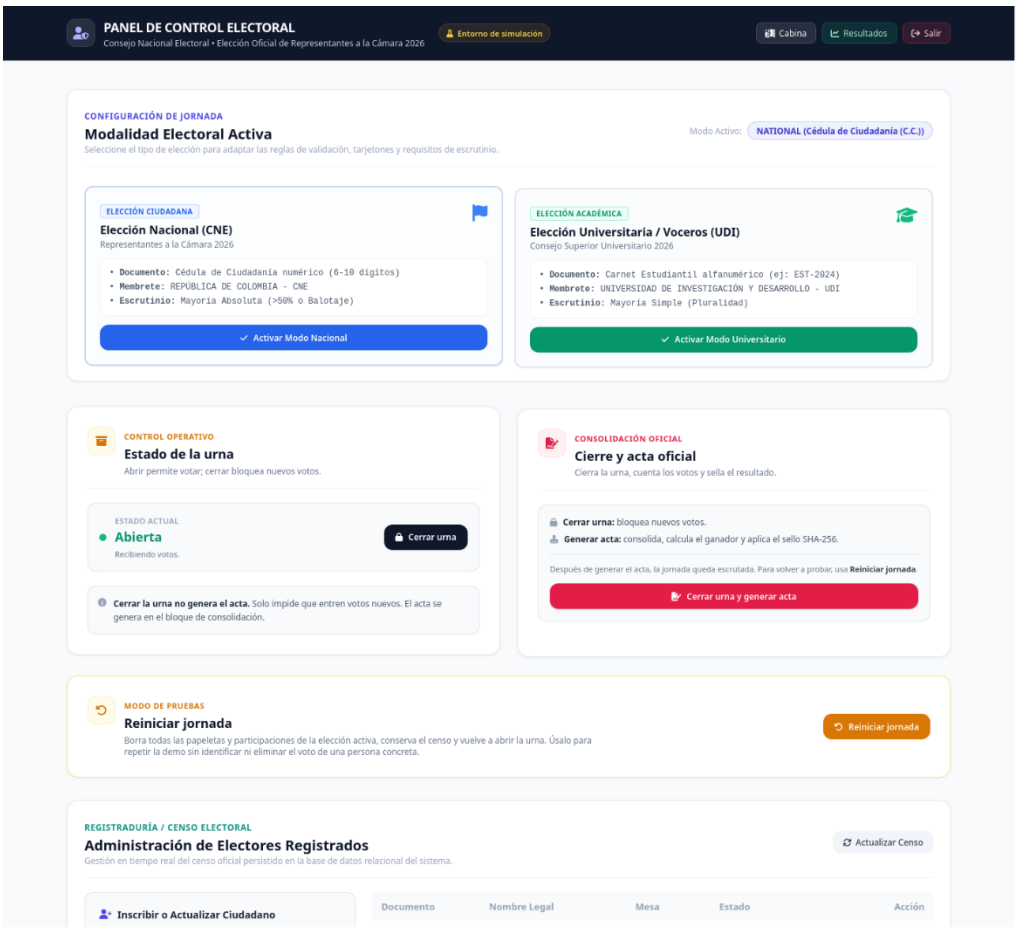


Figura 6. Panel administrativo: estado de la urna, cierre y acta.

3.3 Actualización automática

- La cabina consulta el estado de la urna cada **5 segundos** y la modalidad cada **10 segundos**. Si el administrador cierra la urna, el botón de verificación se deshabilita y cambia a “Urna cerrada”.
- El dashboard se refresca cada **5 segundos** y cambia su botón de cabina por “Urna cerrada” cuando la jornada está bloqueada.
- El panel administrativo refresca estado, censo y bitácora cada **10 segundos**, sin necesidad de pulsar “Actualizar”.

3.4 Qué muestra el acta

Al generar el acta, el panel muestra:

- Total de sufragios.
- Votos por opción.
- Votos en blanco.
- Veredicto según la regla de la modalidad.
- Sello SHA-256 del acta.
- Firmas de jurados de la simulación.

4. Solución de problemas

Síntoma	Causa probable	Solución
La página no carga	El servidor no está corriendo	Ejecutar `python3 -m src.main` desde la raíz.
`ModuleNotFoundError: src`	Se ejecutó `python src/main.py`	Usar `python3 -m src.main`.
El elector no puede votar	PIN incorrecto, documento inhabilitado o ya votó	Revisar el censo en el panel; usar otro documento o reiniciar la jornada.
El panel redirige al login	Sesión vencida o cookie eliminada	Volver a iniciar sesión.
El dashboard muestra 0 votos	La jornada se reinició o no hay votos	Emitir votos de prueba o ejecutar el script de demostración.
Error de base de datos bloqueada	Demasiadas escrituras simultáneas en SQLite	Reintentar; para producción se recomienda PostgreSQL.

Verificación rápida: `pytest -q`

Patrón de Diseño Singleton

1. Identificación del patrón

- **Patrón de Diseño:** Singleton (Creacional).
- **Elemento del Sistema:** `ElectoralConfigManager` (Núcleo del Dominio).
- **Ubicación del Código:** `src/core/config.py`.

2. Justificación y Problema que Resuelve en el Dominio

En un sistema de votación electrónica, la jornada electoral cuenta con un conjunto de reglas y estados globales críticos:

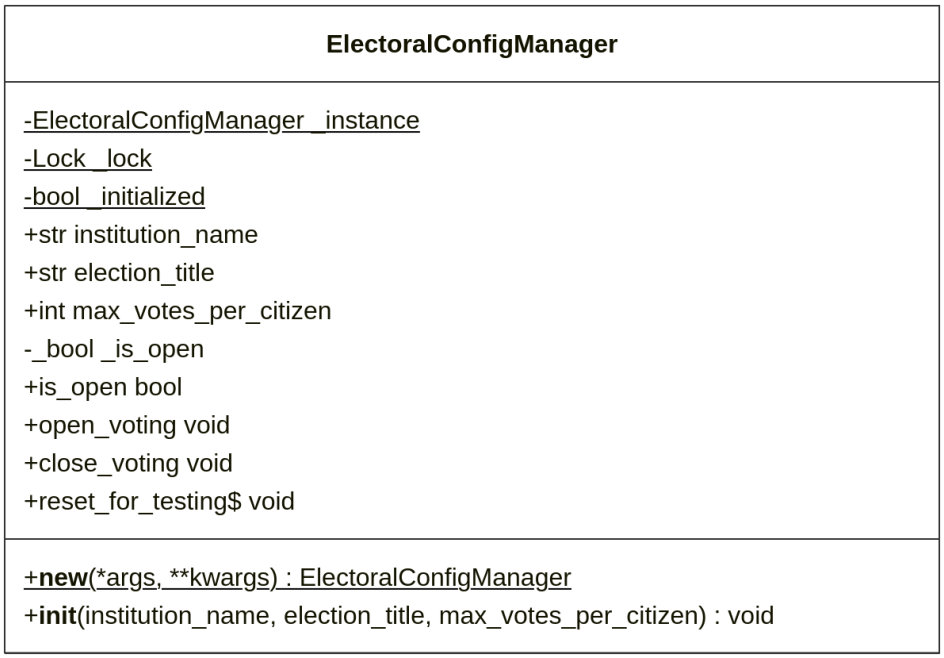
- Estado de la urna electoral (si la votación está abierta o cerrada).
- Nombre oficial de la jornada e institución organizadora.
- Límite legal de votos permitidos por ciudadano.

Si los distintos módulos del sistema (como autenticación de votantes, emisión de sufragios o conteo de votos) gestionaran instancias independientes de la configuración, se producirían inconsistencias de estado. Por ejemplo: el módulo de autenticación podría considerar que la urna sigue abierta mientras que el módulo administrativo ya la cerró.

El patrón **Singleton** garantiza la existencia de **una única instancia global** de la configuración electoral en memoria compartida, proporcionando un punto de acceso centralizado para todos los subsistemas.

3. Modelo Estructural y Diagrama de Clases (UML)

El siguiente diagrama modela la estructura de la clase `ElectoralConfigManager`, destacando los miembros estáticos encargados de controlar la unicidad de instancia (`_instance`), la sincronización en ambientes multihilo (`_lock`) y la prevención de sobreescritura accidental (`_initialized`), así como sus operaciones públicas de control:



4. Implementación Técnica en el Código

En Python, el control de la instanciación única se implementa interceptando el método especial `__new__`, garantizando además la concurrencia mediante cerrojos de sincronización (`threading.Lock`).

4.1 Fragmento de Código Clave (`src/core/config.py`):

```
import threading
from typing import Optional

class ElectoralConfigManager:
    """Gestiona la configuración global de la jornada electoral garantizando una
única instancia."""

    _instance: Optional["ElectoralConfigManager"] = None
    _lock: threading.Lock = threading.Lock()
    _initialized: bool = False

    def __new__(cls, *args, **kwargs) -> "ElectoralConfigManager":
        if cls._instance is None:
            with cls._lock:
                if cls._instance is None:
                    cls._instance = super().__new__(cls)
        return cls._instance

    def __init__(
        self,
        institution_name: str = "Consejo Nacional Electoral",
        election_title: str = "Elecciones Generales",
        max_votes_per_citizen: int = 1,
    ) -> None:
        if self._initialized:
            return

        with self._lock:
            if not self._initialized:
                self.institution_name = institution_name
                self.election_title = election_title
                self.max_votes_per_citizen = max_votes_per_citizen
                self._is_open: bool = False
                self._initialized = True

    @property
    def is_open(self) -> bool:
        return self._is_open

    def open_voting(self) -> None:
        self._is_open = True

    def close_voting(self) -> None:
        self._is_open = False

    @classmethod
    def reset_for_testing(cls) -> None:
        """Permite reiniciar la instancia al reconstruir el sistema o en
pruebas."""
        with cls._lock:
            cls._instance = None
            cls._initialized = False
```

5. Análisis de Ingeniería de Software

5.1 Seguridad en Hilos (*Thread-Safety*)

Se implementó doble comprobación con bloqueo, o double-checked locking.

1. Se verifica si la variable `_instance` es nula antes de adquirir el cerrojo.
2. Si es nula, se entra al bloque `with cls._lock` y se vuelve a comprobar antes de asignar `super().__new__(cls)`.
3. Esto evita el costo de sincronización en llamadas posteriores una vez creada la instancia.

5.2 Control de Re-inicialización

En Python, cada vez que se llama al constructor de clase, `__init__` se invoca de nuevo. Para evitar que llamadas posteriores sobreescriban los datos ya configurados de la elección, se utiliza la bandera `_initialized`.

5.3 Relación con Principios SOLID

- **Single Responsibility Principle (SRP):** La clase asume la responsabilidad exclusiva de salvaguardar y exponer los parámetros centrales de la jornada electoral, centralizando su ciclo de vida.
- **Aislamiento para Pruebas y Reconstrucción (Testability):** Se incorporó `reset_for_testing` para aislar pruebas y para reconstruir la instancia cuando se cambia de modalidad.

Patrón de Diseño Factory Method

1. Identificación del patrón

- **Patrón de Diseño:** Factory Method (Creacional).
- **Elemento del Sistema:** Jerarquía de Papeletas Electorales (`Ballot`) y Emisores (`BallotCreator`).
- **Ubicación del Código:** `src/modules/voting/ballot_factory.py`.

2. Justificación y Problema que Resuelve en el Dominio

Un sistema electoral moderno no maneja un único tipo de voto. En una misma plataforma o jornada pueden coexistir distintos tipos de papeletas de votación:

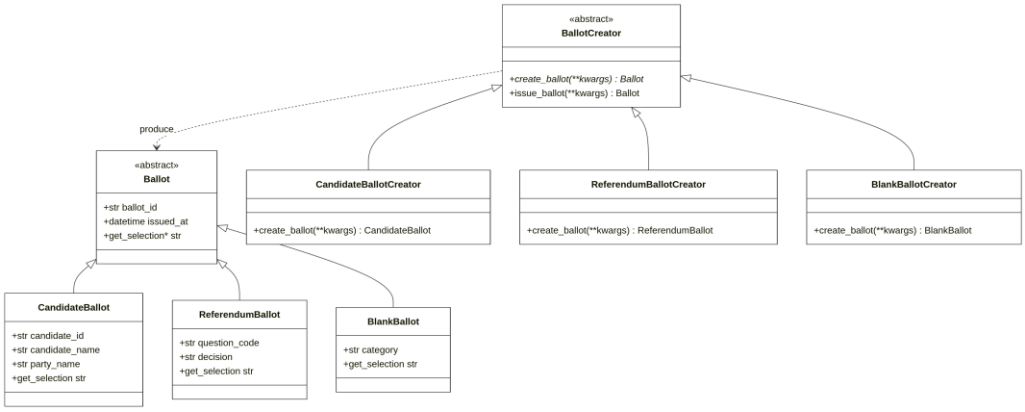
1. **Papeleta por Candidato Nominal:** Contiene identificación del postulante, nombre y partido político.
2. **Papeleta de Referéndum o Consulta Popular:** Contiene el código de la pregunta sometida a votación y la respuesta del ciudadano (SÍ, NO, ABSTENCIÓN).
3. **Papeleta de Voto en Blanco:** Reconocida jurídicamente, sin candidatos asignados pero sujeta a escrutinio formal.

Si el módulo de votación creara estas papeletas directamente mediante condicionales `if/else` o con `new` esparcidos por la lógica de negocio, se violaría el principio **Abierto/Cerrado (OCP)**. Cada vez que se incorporase una nueva modalidad (por ejemplo, voto con lista cerrada preferente), habría que modificar clases existentes y arriesgar la estabilidad del sistema.

El patrón **Factory Method** resuelve este problema definiendo una interfaz común para la creación de un objeto, delegando en subclases creadoras específicas la decisión de qué producto concreto instanciar.

3. Modelo Estructural y Diagrama de Clases (UML)

El siguiente diagrama modela la separación entre la jerarquía de los productos abstractos y concretos (`Ballot`) y la jerarquía de los creadores (`BallotCreator`), mostrando cómo cada fábrica concreta produce su correspondiente tipo de papeleta respetando el principio Open/Closed (OCP):



4. Implementación Técnica en el Código

La implementación separa rigurosamente la jerarquía del **Producto** (`Ballot`) de la jerarquía del **Creador** (`BallotCreator`).

4.1 Jerarquía del Producto (`Ballot` y subclases)

```
from abc import ABC, abstractmethod
from dataclasses import dataclass
from datetime import datetime, timezone
```

```
@dataclass(frozen=True)
class Ballot(ABC):
    """Producto abstracto base para cualquier sufragio emitido."""
    ballot_id: str
    issued_at: datetime

    @abstractmethod
    def get_selection(self) -> str:
        """Devuelve la opción seleccionada de forma legible."""
        pass

@dataclass(frozen=True)
class CandidateBallot(Ballot):
    """Papeleta emitida para un candidato o fórmula nominal."""
    candidate_id: str
    candidate_name: str
    party_name: str

    def get_selection(self) -> str:
        return f"{self.candidate_name} ({self.party_name})"

@dataclass(frozen=True)
class ReferendumBallot(Ballot):
    """Papeleta para consultas populares o referéndums temáticos."""
    question_code: str
    decision: str

    def get_selection(self) -> str:
        return f"[{self.question_code}] {self.decision.upper()}"

@dataclass(frozen=True)
class BlankBallot(Ballot):
    """Papeleta en blanco con validez legal."""
    category: str

    def get_selection(self) -> str:
        return f"VOTO EN BLANCO - {self.category.upper()}"
```

4.2 Jerarquía del Creador (`BallotCreator` y subclases)

```
import uuid

class BallotCreator(ABC):
    """Creador abstracto que define el Factory Method y la lógica de emisión."""

    @abstractmethod
    def create_ballot(self, **kwargs) -> Ballot:
        """Factory Method puro a implementar por cada creador concreto."""
        pass

    def issue_ballot(self, **kwargs) -> Ballot:
        """Método de negocio desacoplado de los detalles de instanciación."""
        ballot = self.create_ballot(**kwargs)
        return ballot

class CandidateBallotCreator(BallotCreator):
    def create_ballot(self, **kwargs) -> CandidateBallot:
        return CandidateBallot(
```

```
            ballot_id=str(uuid.uuid4()),
            issued_at=datetime.now(timezone.utc),
            candidate_id=kwargs["candidate_id"],
            candidate_name=kwargs["candidate_name"],
            party_name=kwargs.get("party_name", "Independiente"),
        )

class ReferendumBallotCreator(BallotCreator):
    def create_ballot(self, **kwargs) -> ReferendumBallot:
        return ReferendumBallot(
            ballot_id=str(uuid.uuid4()),
            issued_at=datetime.now(timezone.utc),
            question_code=kwargs["question_code"],
            decision=kwargs["decision"],
        )

class BlankBallotCreator(BallotCreator):
    def create_ballot(self, **kwargs) -> BlankBallot:
        return BlankBallot(
            ballot_id=str(uuid.uuid4()),
            issued_at=datetime.now(timezone.utc),
            category=kwargs.get("category", "General"),
        )
```

5. Análisis de Ingeniería de Software

5.1 Principio Abierto/Cerrado (OCP)

El sistema queda **abierto a la extensión pero cerrado a la modificación**: si el día de mañana la normativa electoral exige un nuevo tipo de tarjetón (ej. voto para consulta indígena), solo se crean dos clases: `IndigenousBallot` y `IndigenousBallotCreator`. No se modifica ni una sola línea del código existente.

5.2 Inmutabilidad de los Sufragios

Los productos heredan de `@dataclass(frozen=True)`. Esto asegura que una vez emitido el voto, sus atributos no puedan ser alterados en memoria, alineándose con el principio de integridad del sufragio.

5.3 Desacoplamiento de la Operación de Negocio

El método `issue_ballot` en la clase base abstracta utiliza el producto retornado por `create_ballot`, operando únicamente contra la abstracción `Ballot`.

Patrón de Diseño Abstract Factory

1. Identificación del patrón

- **Patrón de Diseño:** Abstract Factory (Creacional).
- **Elemento del Sistema:** Familias de Componentes Electorales (`ElectoralFamilyFactory`).
- **Ubicación del Código:** `src/modules/election/election_factory.py`.

2. Justificación y Problema que Resuelve en el Dominio

Un proceso electoral no consiste únicamente en la emisión aislada de un voto; comprende una serie de componentes dependientes que deben guardar coherencia jurídica y operativa entre sí según la jurisdicción o modalidad de la elección:

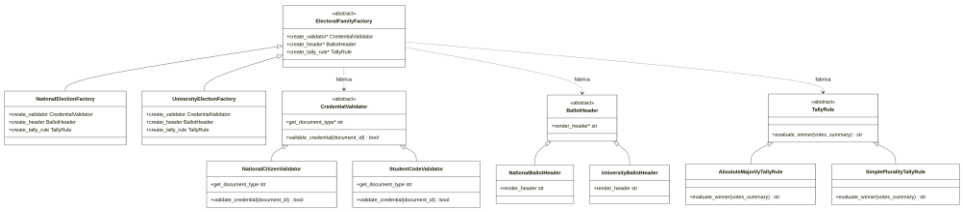
1. **Validación de Identidad (`CredentialValidator`):** Un voto nacional requiere cédula de ciudadanía, mientras que una elección universitaria requiere código o carnet estudiantil.
2. **Encabezado Institucional de la Papeleta (`BallotHeader`):** Cada tipo de elección requiere un membrete oficial específico (Registraduría Nacional vs Consejo Superior Universitario).
3. **Regla de Escrutinio (`TallyRule`):** Una elección presidencial puede exigir una mayoría absoluta (>50%) o determinar balotaje (segunda vuelta), mientras que una elección universitaria se rige habitualmente por mayoría simple (pluralidad de votos).

Si el sistema permitiera instanciar estos componentes de forma arbitraria y cruzada (por ejemplo, validar un carnet estudiantil en una papeleta con el escudo nacional y reglas de mayoría absoluta presidencial), se generarían incongruencias lógicas y jurídicas graves.

El patrón **Abstract Factory** proporciona una interfaz única para crear **familias completas de objetos relacionados o dependientes** sin especificar sus clases concretas, garantizando que todos los elementos utilizados pertenezcan a la misma modalidad electoral.

3. Modelo Estructural y Diagrama de Clases (UML)

El siguiente diagrama modela la arquitectura del patrón Abstract Factory, mostrando la fábrica abstracta (`ElectoralFamilyFactory`), las fábricas concretas por ámbito (`NationalElectionFactory` y `UniversityElectionFactory`), y las tres familias de productos coordinados (`CredentialValidator`, `BallotHeader` y `TallyRule`):



4. Implementación Técnica en el Código

4.1 Productos Abstractos y Familias Concretas

```
from abc import ABC, abstractmethod
from typing import Dict

# --- Producto A: Validador de Credencial ---
class CredentialValidator(ABC):
    @abstractmethod
    def validate_credential(self, document_id: str) -> bool:
        pass
```

```
    @abstractmethod
    def get_document_type(self) -> str:
        pass

class NationalCitizenValidator(CredentialValidator):
    def validate_credential(self, document_id: str) -> bool:
        return document_id.isdigit() and 6 <= len(document_id) <= 10

    def get_document_type(self) -> str:
        return "Cédula de Ciudadanía (C.C.)"

class StudentCodeValidator(CredentialValidator):
    def validate_credential(self, document_id: str) -> bool:
        return len(document_id) >= 5 and any(c.isdigit() for c in document_id)

    def get_document_type(self) -> str:
        return "Código de Carnet Estudiantil"

# --- Producto B: Encabezado de Papeleta ---
class BallotHeader(ABC):
    @abstractmethod
    def render_header(self) -> str:
        pass

class NationalBallotHeader(BallotHeader):
    def render_header(self) -> str:
        return "REPÚBLICA DE COLOMBIA - REGISTRADURÍA NACIONAL DEL ESTADO CIVIL"

class UniversityBallotHeader(BallotHeader):
    def render_header(self) -> str:
        return "UNIVERSIDAD DE INVESTIGACIÓN Y DESARROLLO - CONSEJO SUPERIOR"

# --- Producto C: Regla de Escrutinio ---
class TallyRule(ABC):
    @abstractmethod
    def evaluate_winner(self, votes_summary: Dict[str, int]) -> str:
        pass

class AbsoluteMajorityTallyRule(TallyRule):
    def evaluate_winner(self, votes_summary: Dict[str, int]) -> str:
        total = sum(votes_summary.values())
        if total == 0:
            return "Sin votos registrados"
        for option, count in votes_summary.items():
            if count > total / 2:
                return f"Ganador directo por Mayoría Absoluta (>50%): {option}"
        return "No hay mayoría absoluta: Requiere Segunda Vuelta (Balotaje)"

class SimplePluralityTallyRule(TallyRule):
    def evaluate_winner(self, votes_summary: Dict[str, int]) -> str:
        if not votes_summary:
            return "Sin votos registrados"
        winner = max(votes_summary, key=votes_summary.get)
        return f"Ganador electo por Mayoría Simple: {winner}
({votes_summary[winner]} votos)"
```

4.2 Fábrica Abstracta y Fábricas Concretas

```
class ElectoralFamilyFactory(ABC):
    """Interfaz de la fábrica abstracta para familias de componentes
electorales."""

    @abstractmethod
    def create_validator(self) -> CredentialValidator:
        pass

    @abstractmethod
    def create_header(self) -> BallotHeader:
        pass

    @abstractmethod
    def create_tally_rule(self) -> TallyRule:
        pass

class NationalElectionFactory(ElectoralFamilyFactory):
    """Produce exclusivamente la familia nacional de componentes."""
    def create_validator(self) -> CredentialValidator:
        return NationalCitizenValidator()

    def create_header(self) -> BallotHeader:
        return NationalBallotHeader()

    def create_tally_rule(self) -> TallyRule:
        return AbsoluteMajorityTallyRule()

class UniversityElectionFactory(ElectoralFamilyFactory):
    """Produce exclusivamente la familia universitaria de componentes."""
    def create_validator(self) -> CredentialValidator:
        return StudentCodeValidator()

    def create_header(self) -> BallotHeader:
        return UniversityBallotHeader()

    def create_tally_rule(self) -> TallyRule:
        return SimplePluralityTallyRule()
```

5. Análisis de Ingeniería de Software

5.1 Coherencia de Productos

El mayor beneficio de Abstract Factory en este dominio es garantizar que nunca se mezclen productos de ámbitos distintos. El cliente simplemente solicita una `ElectoralFamilyFactory` y recibe objetos que están garantizados para operar en armonía conceptual y técnica.

5.2 Diferencia Formal con Factory Method

Criterio	Factory Method	Abstract Factory
Alcance	Crea **un solo producto** (`Ballot`).	Crea **familias completas de productos relacionados** (`Validator`, `Header`, `TallyRule`).
Mecanismo	Herencia de clases (`create_ballot`).	Composición de interfaces a nivel de objeto.

Criterio	Factory Method	Abstract Factory
Extensibilidad	Se extiende añadiendo una subclase creadora.	Se extiende añadiendo una nueva fábrica de familia completa.

5.3 Relación con SOLID

- **Open/Closed Principle (OCP):** Es posible introducir una nueva modalidad electoral (por ejemplo, `UnionElectionFactory` para sindicatos) implementando la interfaz abstracta, sin modificar el código que consume los componentes.
- **Dependency Inversion Principle (DIP):** El subsistema que orquesta la votación depende únicamente de la abstracción `ElectoralFamilyFactory` y de las interfaces abstractas de cada producto, nunca de sus clases concretas.

Patrón de Diseño Builder

1. Identificación del patrón

- **Patrón de Diseño:** Builder (Creacional).
- **Elemento del Sistema:** Ensamble de Actas Oficiales de Escrutinio (`ElectoralTallyReport`) mediante `TallyReportBuilder` y `OfficialTallyReportBuilder`.
- **Ubicación del Código:** `src/modules/tally/report_builder.py`.

2. Justificación y Problema que Resuelve en el Dominio

Al finalizar una jornada de votación electrónica, el sistema debe generar el **Acta Oficial de Escrutinio y Cierre de Urnas**. Este objeto no es una estructura de datos simple; se trata de una entidad compleja compuesta por:

- Metadatos de la jornada (título oficial de la elección y jurisdicción territorial).
- Fecha y hora exacta de emisión.
- Lote de identificadores de las mesas de votación escrutadas y consolidadas.
- Desglose numérico de votos válidos por candidato, votos en blanco y votos nulos.
- Cómputo global y totalizado de sufragios.
- Registro de firmas o códigos de autenticación de los jurados de mesa.
- Sello de integridad SHA-256 generado sobre el contenido del acta para detectar alteraciones.

2.1 El Antipatrón Evitado: Constructor Telescópico (*Telescoping Constructor*)

Si intentáramos construir este objeto directamente mediante el constructor de clase `__init__`, caeríamos en el antipatrón de constructor telescópico:

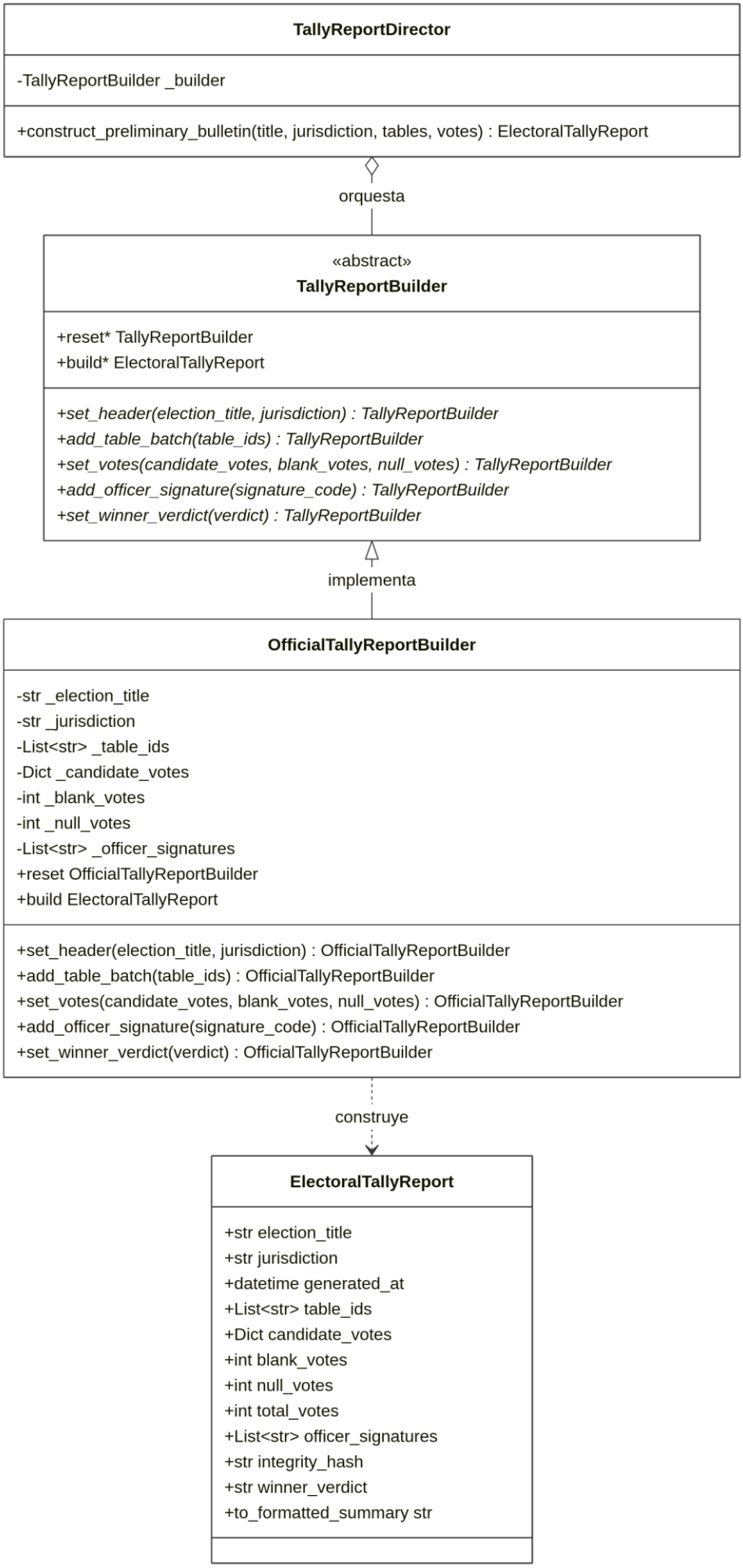
```
# ANTIPATRÓN: Parámetros confusos, propensos a error de orden y difíciles de
mantener
reporte = ElectoralTallyReport("Elecciones 2026", "Nacional", datetime.now, ["M-
01", "M-02"], {...}, 10, 5, 1200, ["JUR-1", "JUR-2"], "a8fbc...")
```

El cliente se vería forzado a conocer el orden exacto de decenas de argumentos, muchos de ellos opcionales o dependientes de pasos de cálculo intermedios.

El patrón **Builder** separa el proceso de construcción paso a paso de un objeto complejo de su representación final, permitiendo que el mismo proceso de construcción produzca representaciones consolidadas.

3. Modelo Estructural y Diagrama de Clases (UML)

El siguiente diagrama modela la separación entre el proceso de construcción progresivo (`TallyReportBuilder`), su implementación concreta (`OfficialTallyReportBuilder`), el director opcional para boletines estándar (`TallyReportDirector`) y el producto final complejo sellado con hash (`ElectoralTallyReport`):


TallyReportDirector
-TallyReportBuilder _builder
+construct_preliminary_bulletin(title, jurisdiction, tables, votes) : ElectoralTallyReport
orquesta
«abstract»
TallyReportBuilder
+reset* TallyReportBuilder
+build* ElectoralTallyReport
+set_header(election_title, jurisdiction) : TallyReportBuilder
+add_table_batch(table_ids) : TallyReportBuilder
+set_votes(candidate_votes, blank_votes, null_votes) : TallyReportBuilder
+add_officer_signature(signature_code) : TallyReportBuilder
+set_winner_verdict(verdict) : TallyReportBuilder
implementa
OfficialTallyReportBuilder
-str _election_title
-str _jurisdiction
-List<str> _table_ids
-Dict _candidate_votes
-int _blank_votes
-int _null_votes
-List<str> _officer_signatures
+reset OfficialTallyReportBuilder
+build ElectoralTallyReport
+set_header(election_title, jurisdiction) : OfficialTallyReportBuilder
+add_table_batch(table_ids) : OfficialTallyReportBuilder
+set_votes(candidate_votes, blank_votes, null_votes) : OfficialTallyReportBuilder
+add_officer_signature(signature_code) : OfficialTallyReportBuilder
+set_winner_verdict(verdict) : OfficialTallyReportBuilder
construye
ElectoralTallyReport
+str election_title
+str jurisdiction
+datetime generated_at
+List<str> table_ids
+Dict candidate_votes
+int blank_votes
+int null_votes
+int total_votes
+List<str> officer_signatures
+str integrity_hash
+str winner_verdict
+to_formatted_summary str

4. Implementación Técnica en el Código

La solución adopta la variante moderna **Fluent Builder** (encadenamiento de métodos) complementada con inmutabilidad en el producto final (`@dataclass(frozen=True)`).

4.1 Producto complejo con campos de solo lectura (`ElectoralTallyReport`)

```
@dataclass(frozen=True)
class ElectoralTallyReport:
    """Producto complejo del acta final de escrutinio (campos de solo lectura)."""
    election_title: str
    jurisdiction: str
    generated_at: datetime
    table_ids: List[str]
    candidate_votes: Dict[str, int]
    blank_votes: int
    null_votes: int
    total_votes: int
    officer_signatures: List[str]
    integrity_hash: str
    winner_verdict: Optional[str] = None

    def to_formatted_summary(self) -> str:
        # Formateo legible y legal del acta para auditoría
        ...
```

4.2 Constructor Abstracto y Concreto (`TallyReportBuilder` / `OfficialTallyReportBuilder`)

```
from datetime import datetime, timezone
from typing import Optional
import hashlib
import json

class TallyReportBuilder(ABC):
    @abstractmethod
    def reset(self) -> "TallyReportBuilder": pass

    @abstractmethod
    def set_header(self, election_title: str, jurisdiction: str) ->
"TallyReportBuilder": pass

    @abstractmethod
    def add_table_batch(self, table_ids: List[str]) -> "TallyReportBuilder": pass

    @abstractmethod
    def set_votes(self, candidate_votes: Dict[str, int], blank_votes: int,
null_votes: int) -> "TallyReportBuilder": pass

    @abstractmethod
    def add_officer_signature(self, signature_code: str) -> "TallyReportBuilder":
pass

    @abstractmethod
    def set_winner_verdict(self, verdict: str) -> "TallyReportBuilder": pass

    @abstractmethod
    def build(self) -> ElectoralTallyReport: pass

class OfficialTallyReportBuilder(TallyReportBuilder):
```

```
    def __init__(self) -> None:
        self.reset()

    def reset(self) -> "OfficialTallyReportBuilder":
        self._election_title = "Jornada Electoral"
        self._jurisdiction = "Nacional"
        self._table_ids = []
        self._candidate_votes = {}
        self._blank_votes = 0
        self._null_votes = 0
        self._officer_signatures = []
        self._winner_verdict: Optional[str] = None
        return self

    def set_header(self, election_title: str, jurisdiction: str) ->
"OfficialTallyReportBuilder":
        self._election_title = election_title
        self._jurisdiction = jurisdiction
        return self

    def add_table_batch(self, table_ids: List[str]) ->
"OfficialTallyReportBuilder":
        self._table_ids.extend(table_ids)
        return self

    def set_votes(self, candidate_votes: Dict[str, int], blank_votes: int,
null_votes: int) -> "OfficialTallyReportBuilder":
        self._candidate_votes = dict(candidate_votes)
        self._blank_votes = max(0, blank_votes)
        self._null_votes = max(0, null_votes)
        return self

    def add_officer_signature(self, signature_code: str) ->
"OfficialTallyReportBuilder":
        if signature_code not in self._officer_signatures:
            self._officer_signatures.append(signature_code)
        return self

    def set_winner_verdict(self, verdict: str) -> "OfficialTallyReportBuilder":
        self._winner_verdict = verdict
        return self

    def build(self) -> ElectoralTallyReport:
        total = sum(self._candidate_votes.values()) + self._blank_votes +
self._null_votes
        now = datetime.now(timezone.utc)

        # Sellado criptográfico automático del estado final
        payload = {
            "title": self._election_title,
            "jurisdiction": self._jurisdiction,
            "timestamp": now.isoformat(),
            "tables": sorted(self._table_ids),
            "candidate_votes": self._candidate_votes,
            "blank": self._blank_votes,
            "null": self._null_votes,
            "total": total,
            "signatures": self._officer_signatures,
            "winner_verdict": self._winner_verdict,
        }
        hash_digest = hashlib.sha256(json.dumps(payload,
sort_keys=True).encode("utf-8")).hexdigest()
```

```
        return ElectoralTallyReport(
            election_title=self._election_title,
            jurisdiction=self._jurisdiction,
            generated_at=now,
            table_ids=list(self._table_ids),
            candidate_votes=dict(self._candidate_votes),
            blank_votes=self._blank_votes,
            null_votes=self._null_votes,
            total_votes=total,
            officer_signatures=list(self._officer_signatures),
            integrity_hash=hash_digest,
            winner_verdict=self._winner_verdict,
        )
```

4.3 Director de Construcción (`TallyReportDirector`) — opcional

Permite automatizar variantes predefinidas del producto, como un boletín preliminar. En el flujo actual de la plataforma no se utiliza; el acta se arma directamente con el builder.

```
class TallyReportDirector:
    def __init__(self, builder: TallyReportBuilder) -> None:
        self._builder = builder

    def construct_preliminary_bulletin(
        self, title: str, jurisdiction: str, tables: List[str], votes: Dict[str,
int]
    ) -> ElectoralTallyReport:
        return (
            self._builder.reset()
            .set_header(f"Boletín Preliminar - {title}", jurisdiction)
            .add_table_batch(tables)
            .set_votes(candidate_votes=votes, blank_votes=0, null_votes=0)
            .build()
        )
```

5. Análisis de Ingeniería de Software

5.1 Inmutabilidad parcial del acta

`frozen=True` impide reasignar los campos del acta, pero las listas y diccionarios internos siguen siendo mutables. Para una inmutabilidad estricta habría que usar tuplas o copias defensivas; hoy el sello SHA-256 permite detectar si el contenido cambió.

5.2 Sello de integridad SHA-256

El método `build` calcula de forma determinista un resumen SHA-256 de todas las partes ensambladas. El acta nace con un sello de integridad verificable; no es una firma digital asimétrica.

5.3 Relación con Principios SOLID

- **Single Responsibility Principle (SRP):** El reporte sólo modela los datos consolidados; el constructor se encarga exclusivamente de las reglas de ensamble, validación de rangos y firmado.
- **Open/Closed Principle (OCP):** Es posible introducir nuevos formatos de acta (por ejemplo, `AuditedJsonTallyReportBuilder` o `InternationalTallyReportBuilder`) implementando `TallyReportBuilder` sin alterar el código cliente existente.

Patrón de Diseño Prototype

1. Identificación del patrón

- **Patrón de Diseño:** Prototype (Creacional).
- **Elemento del Sistema:** Replicación de Mesas y Estaciones de Votación (`VotingStation`) mediante la interfaz genérica `Prototype` y catálogo `StationPrototypeRegistry`.
- **Ubicación del Código:** `src/modules/election/station_prototype.py`.

2. Justificación y Problema que Resuelve en el Dominio

En un despliegue electoral (ya sea a nivel universitario con decenas de mesas, o municipal/nacional con miles de mesas), cada **Mesa o Estación de Votación (`VotingStation`)** comparte una arquitectura operativa base:

- Mismo puesto de votación y código de zona.
- Mismo conjunto de tipos de tarjetones habilitados (presidencial, legislativo, consultas o voto en blanco).
- Mismas políticas de seguridad y estado inicial de habilitación.

Sin embargo, cada mesa física individual debe tener sus propios atributos únicos:

- Identificador unívoco de mesa (ej. `MESA-01`, `MESA-02`).
- Código de hardware o terminal biométrica asignada.
- Lista de jurados asignados para la custodia de la mesa.

2.1 Problema de Creación Directa desde Cero

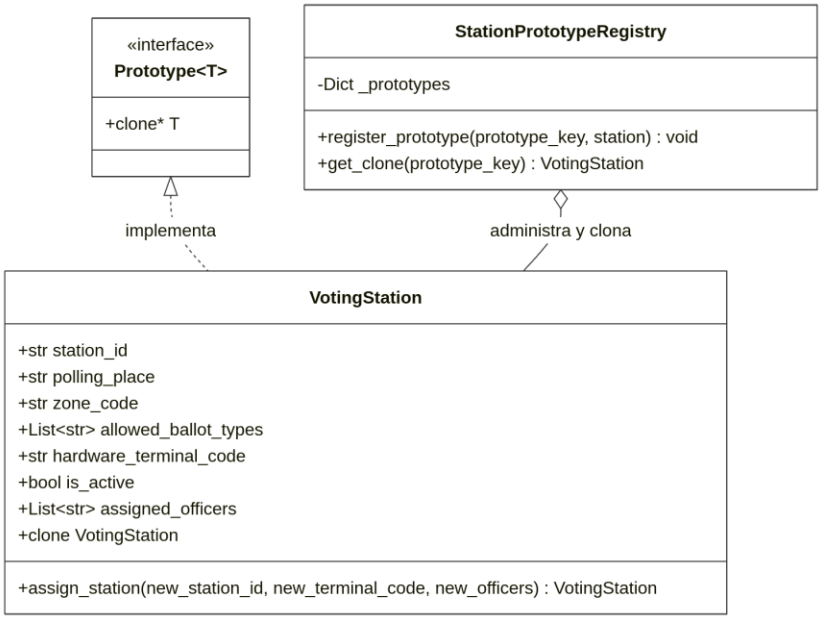
Instanciar repetidamente cada mesa invocando constructores completos con todas las listas y parámetros de configuración:

1. Consume recursos innecesarios recreando estructuras idénticas en memoria.
2. Es propenso a errores humanos de configuración en listas de tarjetones autorizados.
3. Acopla el código de despliegue con la clase concreta de la estación.

El patrón **Prototype** resuelve este problema permitiendo **clonar objetos existentes** para producir réplicas independientes y parametrizables, sin acoplarse a su implementación interna.

3. Modelo Estructural y Diagrama de Clases (UML)

El siguiente diagrama modela la estructura del patrón Prototype, ilustrando el contrato genérico `Prototype[T]`, la entidad clonable `VotingStation` con soporte para personalización mediante `assign_station`, y el catálogo centralizado `StationPrototypeRegistry`:



4. Implementación Técnica en el Código

4.1 La Interfaz Genérica `Prototype`

```
from abc import ABC, abstractmethod
from typing import TypeVar, Generic

T = TypeVar("T", bound="Prototype")

class Prototype(ABC, Generic[T]):
    """Interfaz genérica que formaliza la operación de clonación."""
    @abstractmethod
    def clone(self) -> T:
        """Retorna una copia profunda e independiente del objeto."""
        pass
```

4.2 Entidad Clonable con Copia Profunda (`VotingStation`)

```
import copy
from dataclasses import dataclass, field
from typing import List, Optional

@dataclass
class VotingStation(Prototype["VotingStation"]):
    """Representa una mesa de votación parametrizable y clonable."""
    station_id: str
    polling_place: str
    zone_code: str
    allowed_ballot_types: List[str] = field(default_factory=list)
    hardware_terminal_code: Optional[str] = None
    is_active: bool = True
    assigned_officers: List[str] = field(default_factory=list)

    def clone(self) -> "VotingStation":
        """Genera una copia profunda asegurando total independencia de colecciones
mutables."""
        return copy.deepcopy(self)
```

```
    def assign_station(
        self,
        new_station_id: str,
        new_terminal_code: str,
        new_officers: Optional[List[str]] = None,
    ) -> "VotingStation":
        """Personaliza la estación clonada con identificadores únicos."""
        self.station_id = new_station_id
        self.hardware_terminal_code = new_terminal_code
        if new_officers is not None:
            self.assigned_officers = list(new_officers)
        return self
```

4.3 Catálogo de Prototipos (`StationPrototypeRegistry`)

Permite registrar mesas plantilla según la sede y despachar copias listas para personalizar:

```
from typing import Dict

class StationPrototypeRegistry:
    """Registro de catálogo para almacenar y clonar plantillas de estaciones."""

    def __init__(self) -> None:
        self._prototypes: Dict[str, VotingStation] = {}

    def register_prototype(self, prototype_key: str, station: VotingStation) ->
None:
        self._prototypes[prototype_key] = station

    def get_clone(self, prototype_key: str) -> VotingStation:
        if prototype_key not in self._prototypes:
            raise KeyError(f"Plantilla prototipo '{prototype_key}' no registrada
en el catálogo.")
        return self._prototypes[prototype_key].clone()
```

5. Análisis de Ingeniería de Software

5.1 Análisis Crítico: *Shallow Copy* vs *Deep Copy*

En Python, una copia superficial (`copy.copy`) copia los campos del objeto pero mantiene la misma referencia en memoria para colecciones mutables como listas y diccionarios (`allowed_ballot_types`, `assigned_officers`).

- **Riesgo electoral:** Si se usara copia superficial, agregar un jurado a la Mesa 2 modificaría automáticamente la lista de jurados de la Mesa 1 y de la plantilla prototipo, generando una falla crítica de integridad de datos.
- **Solución aplicada:** Se utiliza estrictamente `copy.deepcopy(self)`, lo que asegura que cada clon posea su propio espacio en memoria completamente aislado.

5.2 Relación con Principios SOLID

- **Single Responsibility Principle (SRP):** Cada estación sabe cómo duplicarse a sí misma de manera segura; el registro (`StationPrototypeRegistry`) se enfoca exclusivamente en la administración de las plantillas.
- **Open/Closed Principle (OCP):** Nuevos tipos de estaciones o configuraciones de mesas pueden incorporarse y registrarse en tiempo de ejecución sin alterar el código del motor electoral.

Patrones de Diseño Adapter y Bridge

1. Identificación del patrón

- **Patrones de Diseño:** Adapter y Bridge (Patrones Estructurales).
- **Elementos del Sistema:**
  - **Módulo de Censo Electoral (Adapter):** Interfaz canónica `ICensusRegistry`, entidad inmutable `VoterCensusRecord`, y adaptadores concretos `JsonCensusAdapter`, `CsvCensusAdapter` y `SqliteCensusAdapter` (fuente activa en la demo).
  - **Módulo de Transmisión Multicanal (Bridge):** Abstracción base `ElectoralBulletin` con sus refinamientos (`PreliminaryBulletin`, `AuditAlertBulletin`, `OfficialClosingBulletin`), conectada mediante puente a la interfaz implementadora `ITransmissionChannel` (`ConsoleTransmissionChannel`, `FileTransmissionChannel`, `MemoryAuditSinkTransmissionChannel`).
- **Ubicación del Código:**
  - `src/modules/census/census_adapter.py`
  - `src/modules/transmission/bulletin_bridge.py`

2. Justificación y Problema que Resuelven en el Dominio

En esta etapa del sistema electoral, el reto principal consiste en **conectar y comunicar componentes heterogéneos sin acoplar la arquitectura**:

2.1 Problemática de Incompatibilidad de Fuentes: Justificación de ADAPTER

Para ejercer el voto, el sistema debe consultar un padrón electoral oficial. Sin embargo, en la práctica gubernamental e institucional, los censos provienen de orígenes completamente dispares:

1. **Elecciones Nacionales:** La Registraduría Nacional expide volcados masivos en formato de texto plano CSV con encabezados legados (`cedula,nombre_apellidos,mesa_num,habilitado_sufragio`).
2. **Elecciones Institucionales o Universitarias:** Las oficinas de registro y control proveen APIs REST o exportaciones JSON estructuradas con llaves diferentes (`codigo,nombres,mesa,activo`).

Si el motor de votación se programara para leer directamente las llaves de un JSON o las columnas de un CSV, se violaría gravemente el principio de inversión de dependencias (DIP) y el principio de responsabilidad única (SRP). Cualquier cambio en la estructura del archivo externo rompería la lógica electoral.

El patrón **Adapter** resuelve esta incompatibilidad actuando como un traductor intermedio: envuelve la estructura externa no estándar (*Adaptee*) y expone la interfaz unificada que el sistema electoral espera (*Target*: `ICensusRegistry`).

2.2 Problemática de Explosión Combinatoria en la Transmisión: Justificación de BRIDGE

Durante el transcurso de las elecciones y al cierre de urnas, se requiere difundir información oficial hacia el exterior. Este flujo varía de manera independiente en **dos dimensiones ortogonales**:

- **Dimensión 1 (El "Qué" se transmite - Abstracción):** Distintos tipos de boletines con información y propósitos disímiles:
  - *Boletín Preliminar:* Avance parcial del conteo de votos en vivo con porcentaje de mesas informadas.
  - *Alerta de Seguridad/Auditoría:* Notificación urgente por incidencias críticas (ej. intento de doble sufragio, alteración de precinto).
  - *Boletín de Cierre Oficial:* Acta definitiva de escrutinio con sello de integridad SHA-256.

- **Dimensión 2 (El "Cómo / Dónde" se despacha - Implementador):** Múltiples medios técnicos o canales de salida:
    - *Consola / Log de Operaciones del Sistema.*
    - *Almacenamiento Local Sellado en Archivo de Disco.*
    - *Receptor de Memoria (Sink) para Auditoría y Testing Automatizado.*

El Antipatrón de Herencia Evitado (Explosión Combinatoria):

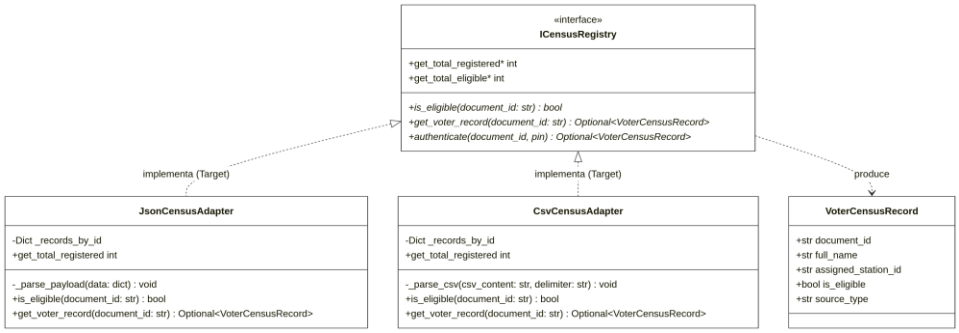
Si resolviéramos esta necesidad mediante herencia tradicional, tendríamos que crear clases para cada combinación posible: `PreliminaryBulletinConsole`, `PreliminaryBulletinFile`, `AuditAlertConsole`, `AuditAlertFile`, `ClosingBulletinConsole`, etc. Con 3 tipos de boletines y 3 canales de salida, se requerirían $3 \times 3 = 9$ **clases**. Si el día de mañana se agrega un nuevo canal (ej. Webhook REST o Transmisión Satelital), se sumarían 3 clases más por cada nuevo medio.

El patrón **Bridge** resuelve este dilema desacoplando la jerarquía de abstracción (los boletines) de la jerarquía de implementación (los canales de transporte), combinándolos por **composición**. Esto reduce el número total de clases a $3 + 3 = 6$, permitiendo que cualquiera de las dos jerarquías crezca de forma autónoma sin impactar a la otra.

3. Parte A: Patrón de Diseño ADAPTER

3.1 Modelo Estructural y Diagrama de Clases (UML)

El siguiente diagrama modela la interfaz objetivo `ICensusRegistry` (Target), el registro canónico de elector `VoterCensusRecord`, y los dos adaptadores de objeto concretos (`JsonCensusAdapter` y `CsvCensusAdapter`):



3.2 Implementación Técnica en el Código (`src/modules/census/census_adapter.py`)

Interfaz Objetivo (Target) y Entidad Canónica:

```
@dataclass(frozen=True)
class VoterCensusRecord:
    """Representación canónica e inmutable de un registro de censo en el
dominio."""
    document_id: str
    full_name: str
    assigned_station_id: str
    is_eligible: bool
    source_type: str

class ICensusRegistry(ABC):
    """Interfaz objetivo (Target) requerida por el dominio para validar el
censo."""

    @abstractmethod
    def is_eligible(self, document_id: str) -> bool:
```

```
        """Determina si un ciudadano está habilitado legalmente para sufragar."""
        pass

    @abstractmethod
    def get_voter_record(self, document_id: str) -> Optional[VoterCensusRecord]:
        """Obtiene la ficha censal canónica del elector."""
        pass

    @abstractmethod
    def get_total_registered(self) -> int:
        """Retorna el número total de votantes registrados en el censo
adaptado."""
        pass

    @abstractmethod
    def get_total_eligible(self) -> int:
        """Retorna cuántos electores están habilitados para votar."""
        pass

    @abstractmethod
    def authenticate(self, document_id: str, pin: str) ->
Optional[VoterCensusRecord]:
        """Valida documento + PIN y devuelve la ficha canónica."""
        pass
```

Adaptador para Esquemas JSON Heterogéneos (`JsonCensusAdapter`):

```
class JsonCensusAdapter(ICensusRegistry):
    """Adaptador de objeto para fuentes de datos en formato JSON."""

    def __init__(self, raw_json_payload: str | Dict[str, Any]) -> None:
        data = json.loads(raw_json_payload) if isinstance(raw_json_payload, str)
else raw_json_payload
        self._records_by_id: Dict[str, VoterCensusRecord] = {}
        self._parse_payload(data)

    def _parse_payload(self, data: Dict[str, Any]) -> None:
        items = data.get("electores", data.get("estudiantes", data.get("citizens",
[])))
        for item in items:
            doc = str(item.get("documento_id", item.get("codigo", item.get("id",
""")))).strip()
            name = item.get("nombre_completo", item.get("nombres",
item.get("name", "Desconocido"))).strip()
            station = item.get("mesa_asignada", item.get("mesa",
item.get("station", "MESA-GENERAL"))).strip()
            eligible = bool(item.get("habilitado", item.get("activo",
item.get("eligible", True))))

            if doc:
                self._records_by_id[doc] = VoterCensusRecord(
                    document_id=doc,
                    full_name=name,
                    assigned_station_id=station,
                    is_eligible=eligible,
                    source_type="JSON_ADAPTER",
                )

    def is_eligible(self, document_id: str) -> bool:
        record = self._records_by_id.get(str(document_id).strip())
```

```
        return record.is_eligible if record else False

    def get_voter_record(self, document_id: str) -> Optional[VoterCensusRecord]:
        return self._records_by_id.get(str(document_id).strip())

    def get_total_registered(self) -> int:
        return len(self._records_by_id)
```

Adaptador para Esquemas Tabulares CSV (`CsvCensusAdapter`):

```
class CsvCensusAdapter(ICensusRegistry):
    """Adaptador de objeto para fuentes tabulares en formato CSV plano."""

    def __init__(self, csv_content: str, delimiter: str = ",") -> None:
        self._records_by_id: Dict[str, VoterCensusRecord] = {}
        self._parse_csv(csv_content, delimiter)

    def _parse_csv(self, csv_content: str, delimiter: str) -> None:
        reader = csv.DictReader(io.StringIO(csv_content.strip()),
delimiter=delimiter)
        for row in reader:
            norm = {k.strip().lower(): v.strip() for k, v in row.items() if k}
            doc = norm.get("cedula") or norm.get("documento") or
norm.get("document_id", "")
            name = norm.get("nombre") or norm.get("nombre_completo") or
norm.get("full_name", "Desconocido")
            station = norm.get("mesa") or norm.get("mesa_asignada") or "MESA-
GENERAL"
            status_raw = (norm.get("habilitado") or norm.get("estado",
"si")).lower()
            eligible = status_raw in ("si", "sí", "true", "1", "habilitado",
"activo")

            if doc:
                self._records_by_id[doc] = VoterCensusRecord(
                    document_id=doc,
                    full_name=name,
                    assigned_station_id=station,
                    is_eligible=eligible,
                    source_type="CSV_ADAPTER",
                )

    def is_eligible(self, document_id: str) -> bool:
        record = self._records_by_id.get(str(document_id).strip())
        return record.is_eligible if record else False

    def get_voter_record(self, document_id: str) -> Optional[VoterCensusRecord]:
        return self._records_by_id.get(str(document_id).strip())

    def get_total_registered(self) -> int:
        return len(self._records_by_id)

    def get_total_eligible(self) -> int:
        return sum(1 for record in self._records_by_id.values() if
record.is_eligible)

    def authenticate(self, document_id: str, pin: str) ->
Optional[VoterCensusRecord]:
        # JSON/CSV no traen PIN verificable: se falla cerrado.
        return None
```

Adaptador SQLite (fuente activa de la demo)

`SqliteCensusAdapter` implementa el mismo Target, pero consulta la tabla `census_citizens` y valida el PIN con `verify_pin`. Es el adaptador que usa hoy la fachada en la demo; JSON y CSV quedan como ejemplos de integración.

```
from src.core.database import CitizenModel, SessionLocal, init_db
from src.core.security import verify_pin

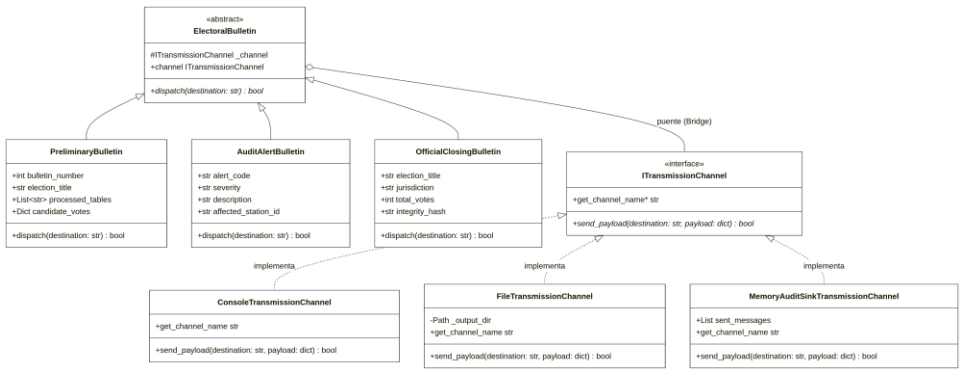
class SqliteCensusAdapter(ICensusRegistry):
    def __init__(self, election_type: str = "NATIONAL") -> None:
        self.election_type = election_type.upper().strip()
        init_db()

    def authenticate(self, document_id: str, pin: str) ->
Optional[VoterCensusRecord]:
        session = SessionLocal()
        try:
            citizen = session.query(CitizenModel).filter(
                CitizenModel.document_id == str(document_id).strip(),
                CitizenModel.election_type == self.election_type,
            ).first()
            if not citizen or not citizen.is_eligible:
                return None
            if not verify_pin(pin, citizen.pin_hash):
                return None
            return VoterCensusRecord(
                document_id=citizen.document_id,
                full_name=citizen.full_name,
                assigned_station_id=citizen.assigned_station_id,
                is_eligible=citizen.is_eligible,
                source_type="SQLITE_DATABASE",
            )
        finally:
            session.close()
```

4. Parte B: Patrón de Diseño BRIDGE

4.1 Modelo Estructural y Diagrama de Clases (UML)

El siguiente diagrama modela la separación ortogonal entre la jerarquía de la Abstracción (`ElectoralBulletin`) y la jerarquía del Implementador (`ITransmissionChannel`), vinculadas por composición a través del puente de referencia:



4.2 Implementación Técnica en el Código (`src/modules/transmission/bulletin_bridge.py`)

Jerarquía del Implementador (Canales Físicos de Salida):

```
class ITransmissionChannel(ABC):
    """Interfaz Implementor que define el contrato de transporte de datos."""

    @abstractmethod
    def send_payload(self, destination: str, payload: Dict[str, Any]) -> bool:
        pass

    @abstractmethod
    def get_channel_name(self) -> str:
        pass

class ConsoleTransmissionChannel(ITransmissionChannel):
    """Implementador concreto para consola/log oficial."""
    def send_payload(self, destination: str, payload: Dict[str, Any]) -> bool:
        timestamp = payload.get("timestamp",
datetime.now(timezone.utc).isoformat())
        print(f"\n[CANAL CONSOLA :: {destination}] ({timestamp}) Tipo:
{payload.get('type')}")
        print(json.dumps(payload, indent=2, ensure_ascii=False))
        return True

    def get_channel_name(self) -> str:
        return "CONSOLE_LOG"

class FileTransmissionChannel(ITransmissionChannel):
    """Implementador concreto que persiste en disco como bitácora sellada."""
    def __init__(self, output_dir: str = "/tmp") -> None:
        self._output_dir = Path(output_dir)
        self._output_dir.mkdir(parents=True, exist_ok=True)

    def send_payload(self, destination: str, payload: Dict[str, Any]) -> bool:
        safe_name = destination.replace("/", "_").replace(" ", "_").lower()
        file_path = self._output_dir / f"transmission_{safe_name}.jsonl"
        with open(file_path, "a", encoding="utf-8") as f:
            f.write(json.dumps(payload, ensure_ascii=False) + "\n")
        return True

    def get_channel_name(self) -> str:
        return f"FILE_STORAGE ({self._output_dir})"

class MemoryAuditSinkTransmissionChannel(ITransmissionChannel):
    """Implementador concreto para auditoría en memoria y pruebas
automatizadas."""
    def __init__(self) -> None:
        self.sent_messages: List[Dict[str, Any]] = []

    def send_payload(self, destination: str, payload: Dict[str, Any]) -> bool:
        self.sent_messages.append({"destination": destination, "payload":
dict(payload)})
        return True

    def get_channel_name(self) -> str:
        return "IN_MEMORY_SINK"
```

Jerarquía de la Abstracción y Refinamientos Electorales:

```
class ElectoralBulletin(ABC):
    """Abstracción base que mantiene una referencia al implementador (Bridge)."""

    def __init__(self, channel: ITransmissionChannel) -> None:
        self._channel: ITransmissionChannel = channel

    @property
    def channel(self) -> ITransmissionChannel:
        return self._channel

    @channel.setter
    def channel(self, new_channel: ITransmissionChannel) -> None:
        """Permite intercambiar el canal de transporte en tiempo de ejecución."""
        self._channel = new_channel

    @abstractmethod
    def dispatch(self, destination: str) -> bool:
        pass

class PreliminaryBulletin(ElectoralBulletin):
    """Abstracción refinada: Boletín de escrutinio parcial en vivo."""
    def __init__(self, channel: ITransmissionChannel, bulletin_number: int,
                 election_title: str, processed_tables: List[str],
                 candidate_votes: Dict[str, int]) -> None:
        super().__init__(channel)
        self.bulletin_number = bulletin_number
        self.election_title = election_title
        self.processed_tables = list(processed_tables)
        self.candidate_votes = dict(candidate_votes)

    def dispatch(self, destination: str) -> bool:
        payload = {
            "type": "PRELIMINARY_BULLETIN",
            "bulletin_number": self.bulletin_number,
            "election_title": self.election_title,
            "timestamp": datetime.now(timezone.utc).isoformat(),
            "tables_processed_count": len(self.processed_tables),
            "tables": self.processed_tables,
            "candidate_votes": self.candidate_votes,
            "total_votes_tallied": sum(self.candidate_votes.values()),
        }
        return self._channel.send_payload(destination=destination,
payload=payload)

class AuditAlertBulletin(ElectoralBulletin):
    """Abstracción refinada: Alerta prioritaria de seguridad o auditoría."""
    def __init__(self, channel: ITransmissionChannel, alert_code: str,
                 severity: str, description: str, affected_station_id: str) ->
None:
        super().__init__(channel)
        self.alert_code = alert_code
        self.severity = severity.upper()
        self.description = description
        self.affected_station_id = affected_station_id

    def dispatch(self, destination: str) -> bool:
        payload = {
            "type": "AUDIT_SECURITY_ALERT",
            "alert_code": self.alert_code,
            "severity": self.severity,
```

```
            "station_id": self.affected_station_id,
            "description": self.description,
            "timestamp": datetime.now(timezone.utc).isoformat(),
        }
        return self._channel.send_payload(destination=destination,
payload=payload)

class OfficialClosingBulletin(ElectoralBulletin):
    """Boletín formal de cierre: acta consolidada con sello SHA-256."""
    def __init__(self, channel, election_title: str, jurisdiction: str,
                 total_votes: int, integrity_hash: str) -> None:
        super().__init__(channel)
        self.election_title = election_title
        self.jurisdiction = jurisdiction
        self.total_votes = total_votes
        self.integrity_hash = integrity_hash

    def dispatch(self, destination: str) -> bool:
        payload = {
            "type": "OFFICIAL_CLOSING_BULLETIN",
            "election_title": self.election_title,
            "jurisdiction": self.jurisdiction,
            "total_votes": self.total_votes,
            "integrity_hash": self.integrity_hash,
            "timestamp": datetime.now(timezone.utc).isoformat(),
        }
        return self._channel.send_payload(destination=destination,
payload=payload)
```

5. Análisis de Ingeniería de Software

5.1 Comparativa Formal: Adapter vs Bridge

ambos patrones emplean envoltura y composición pero responden a intenciones radicalmente diferentes:

Criterio	Patrón ADAPTER	Patrón BRIDGE
Intención GoF	*Convertir la interfaz de una clase en otra interfaz que el cliente espera.*	*Desacoplar una abstracción de su implementación para que varíen independientemente.*
Problema que Resuelve	Compatibilidad a posteriori: clases ya existentes cuyas interfaces no encajan.	Diseño preventivo: evitar la explosión combinatoria multidimensional ($M \times N$).
Momento de Aplicación	Se aplica sobre sistemas existentes o formatos externos (Censo CSV/JSON).	Se planifica desde el inicio de la arquitectura (Despacho multicanal de boletines).
Relación de Interfaces	Adapta una interfaz incompatible a la interfaz esperada por el Target.	Conecta dos jerarquías paralelas diseñadas para colaborar desde el inicio.
Rol en el Sistema	**Inbound (Entrada):** Ingesta y saneamiento de electores.	**Outbound (Salida):** Transmisión y reporte de eventos oficiales.

5.2 Cumplimiento de Principios SOLID

- **Single Responsibility Principle (SRP):**
  - Los adaptadores solo traducen formatos de datos al modelo canónico; no validan urnas ni cuentan votos.
  - La abstracción `ElectoralBulletin` solo modela la semántica del mensaje; el canal `ITransmissionChannel` se ocupa exclusivamente del protocolo de transporte.
- **Open/Closed Principle (OCP):**
  - El proyecto ya incluye `SqliteCensusAdapter` para la fuente activa; si mañana aparece otra fuente (SOAP, REST, etc.), se agrega un adaptador nuevo sin alterar el motor electoral.
  - Si se añade un canal de difusión por Webhook o satélite, solo se implementa `RestWebhookTransmissionChannel` sin tocar los boletines existentes.
- **Dependency Inversion Principle (DIP):**
  - Los casos de uso electorales solo dependen de `ICensusRegistry` y de `ElectoralBulletin`, operando contra abstracciones puras.
- **Liskov Substitution Principle (LSP):**
  - Cualquier canal concreto (`Console`, `File`, `MemorySink`) puede sustituir a otro en tiempo de ejecución mediante la propiedad `bulletin.channel = nuevo_canal` sin alterar la lógica de despacho.

5.3 Testabilidad y Aislamiento para Pruebas

Gracias al canal `MemoryAuditSinkTransmissionChannel`, el equipo de auditoría y la suite de pruebas automatizadas pueden verificar la emisión de boletines y alertas en microsegundos dentro de la memoria RAM, sin necesidad de escribir en disco ni requerir sockets de red activos.

Patrones de Diseño Composite, Decorator y Facade

1. Identificación del patrón

- **Patrones de Diseño:** Composite, Decorator y Facade (Patrones Estructurales).
- **Elementos del Sistema:**
    - **Estructura Territorial y Mesas (Composite):** Componente abstracto `ElectoralUnit`, hoja terminal `VotingStationLeaf` y contenedor jerárquico `ElectoralJurisdictionComposite`.
    - **Seguridad y Trazabilidad del Voto (Decorator):** Decorador base `BallotSecurityDecorator`, y decoradores concretos `TimestampSecurityDecorator`, `DigitalSignatureDecorator` y `AuditWatermarkDecorator`.
    - **Orquestador Unificado del Sistema (Facade):** Fachada central `VotingSystemFacade` que coordina los subsistemas creacionales y estructurales en una interfaz de alto nivel.
- **Ubicación del Código:**
    - `src/modules/election/jurisdiction_composite.py`
    - `src/modules/voting/ballot_decorator.py`
    - `src/modules/facade/voting_facade.py`

2. Justificación y Problema que Resuelven en el Dominio

El sistema integra tres patrones estructurales para resolver necesidades críticas del proceso electoral:

2.1 Justificación de COMPOSITE: Organización Territorial y de Recintos

Un proceso electoral se organiza de forma natural como un árbol jerárquico parte-todo:

- Una **Circunscripción o Institución** se divide en **Zonas o Facultades**.
- Cada Zona agrupa múltiples **Puestos de Votación** (colegios o campus).
- Cada Puesto contiene un número variable de **Mesas de Votación individuales** (`VotingStation`).

Problema que evita: Sin Composite, el cliente tendría que escribir algoritmos diferentes y condicionales `if/else` para consultar cuántas mesas tiene un puesto, cuántas tiene una comuna y cuántas tiene todo el país. **Solución Composite:** Define la interfaz común `ElectoralUnit`. El cliente trata con idéntica uniformidad a una mesa individual (hoja) que a una circunscripción nacional entera (compuesto), propagando operaciones recursivas (`count_stations`, `get_station_ids`, `display_hierarchy`).

2.2 Justificación de DECORATOR: Seguridad Dinámica del Voto

Una papeleta emitida (`Ballot`) almacena la opción elegida por el votante. No obstante, según la legislación de la jornada y los requerimientos de auditoría, el sufragio requiere capas de seguridad añadidas:

1. Sello de tiempo UTC.
2. Sello de integridad HMAC-SHA256 con clave del servicio y terminal de emisión.
3. Código de recibo y marca de agua para verificación del ciudadano.

El Antipatrón Evitado (Explosión de Subclases): Si intentáramos modelar estas combinaciones mediante herencia de clases, tendríamos una explosión de 2^n clases (`BallotConTimestamp`, `BallotConTimestampYFirma`, `BallotConTimestampFirmaYMarca`, etc.). **Solución Decorator:** Envuelve dinámicamente el voto en memoria en tiempo de ejecución. Permite anidar tantas capas de seguridad como la elección demande (`DigitalSignatureDecorator(TimestampSecurityDecorator(ballot))`) sin alterar la entidad base `Ballot` y garantizando que el objeto siga cumpliendo la interfaz de sufragio.

2.3 Justificación de FACADE: La Puerta de Entrada hacia la Interfaz Gráfica

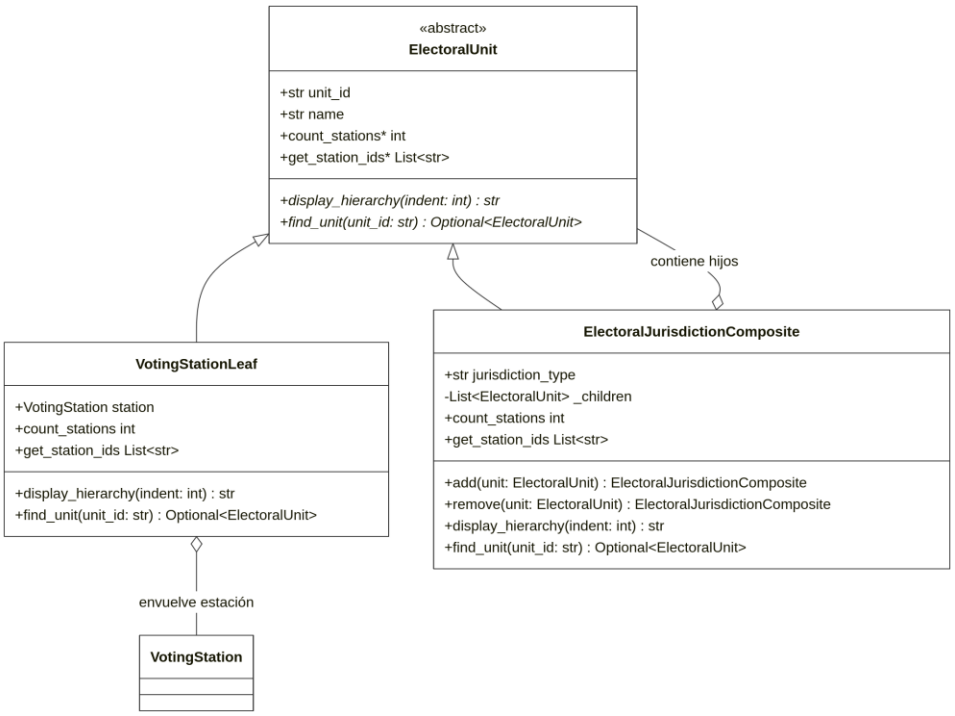
El sistema ya tiene implementados y coordinados los patrones creacionales y estructurales que necesita la fachada. Si la futura capa web (FastAPI, Swagger UI o plantillas HTML) tuviera que interactuar directamente con cada módulo:

- Cada endpoint requeriría decenas de líneas de código instanciando fábricas, adaptadores de censo, constructores de actas y validadores.
- La capa de presentación quedaría fuertemente acoplada a las clases internas del dominio.

Solución Facade: `VotingSystemFacade` expone operaciones de alto nivel (`open_polls`, `verify_voter`, `cast_secure_vote`, `close_and_generate_tally`), encapsulando la complejidad y dejando el sistema listo para conectarse a cualquier interfaz de usuario sin escribir código espagueti.

3. Parte A: Patrón de Diseño COMPOSITE

3.1 Modelo Estructural y Diagrama de Clases (UML)



3.2 Implementación Técnica en Código (`src/modules/election/jurisdiction_composite.py`)

```
class ElectoralUnit(ABC):
    """Componente abstracto base para elementos de la jerarquía territorial
electoral."""

    def __init__(self, unit_id: str, name: str) -> None:
        self.unit_id = unit_id
        self.name = name

    @abstractmethod
    def count_stations(self) -> int:
        pass

    @abstractmethod
    def get_station_ids(self) -> List[str]:
        pass
```

```
class VotingStationLeaf(ElectoralUnit):
    """Hoja (Leaf) que representa una mesa de votación individual terminal."""

    def __init__(self, station: VotingStation) -> None:
        super().__init__(unit_id=station.station_id, name=f"Mesa
{station.station_id}")
        self.station = station

    def count_stations(self) -> int:
        return 1

    def get_station_ids(self) -> List[str]:
        return [self.station.station_id]

class ElectoralJurisdictionComposite(ElectoralUnit):
    """Compuesto (Composite) que agrupa unidades electorales recursivamente."""

    def __init__(self, unit_id: str, name: str, jurisdiction_type: str = "ZONA") -
> None:
        super().__init__(unit_id, name)
        self.jurisdiction_type = jurisdiction_type.upper()
        self._children: List[ElectoralUnit] = []

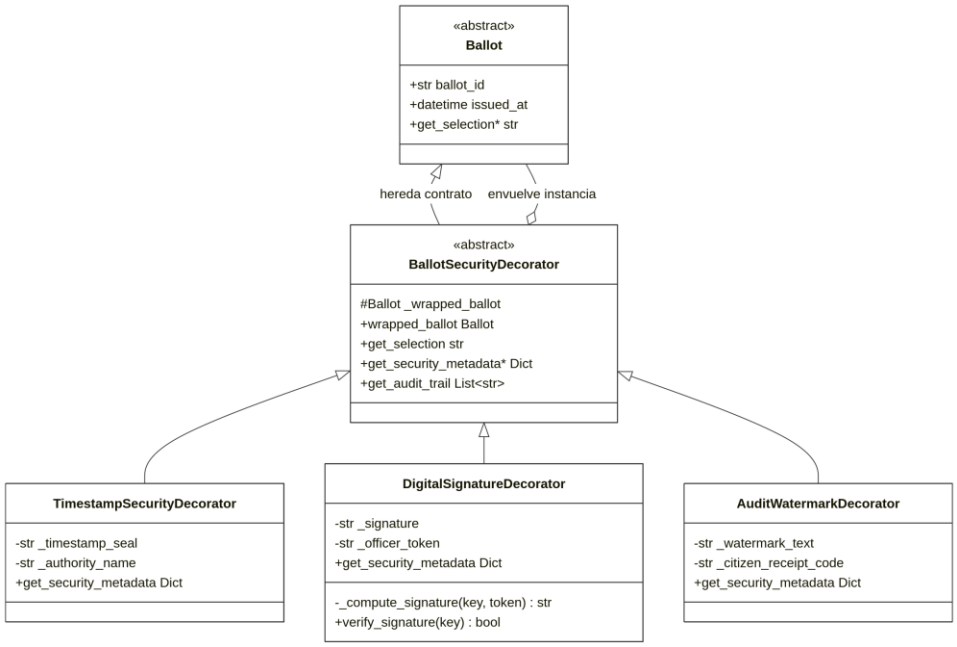
    def add(self, unit: ElectoralUnit) -> ElectoralJurisdictionComposite:
        if unit not in self._children:
            self._children.append(unit)
        return self

    def count_stations(self) -> int:
        return sum(child.count_stations() for child in self._children)

    def get_station_ids(self) -> List[str]:
        station_ids: List[str] = []
        for child in self._children:
            station_ids.extend(child.get_station_ids())
        return station_ids
```

4. Parte B: Patrón de Diseño DECORATOR

4.1 Modelo Estructural y Diagrama de Clases (UML)



4.2 Implementación Técnica en Código (`src/modules/voting/ballot_decorator.py`)

```
class BallotSecurityDecorator(Ballot, ABC):
    """Decorador abstracto base que envuelve una papeleta conservando su contrato."""

    def __init__(self, wrapped_ballot: Ballot) -> None:
        super().__init__(ballot_id=wrapped_ballot.ballot_id,
issued_at=wrapped_ballot.issued_at)
        object.__setattr__(self, "_wrapped_ballot", wrapped_ballot)

    def get_selection(self) -> str:
        return self.wrapped_ballot.get_selection()

    @abstractmethod
    def get_security_metadata(self) -> Dict[str, Any]:
        pass

class DigitalSignatureDecorator(BallotSecurityDecorator):
    """Decorador concreto que sella criptográficamente la papeleta mediante HMAC-
SHA256."""

    def __init__(self, wrapped_ballot: Ballot, signing_key: str, officer_token: str) ->
None:
        super().__init__(wrapped_ballot)
        signature = self._compute_signature(signing_key, officer_token)
        object.__setattr__(self, "_signature", signature)
        object.__setattr__(self, "_officer_token", officer_token)

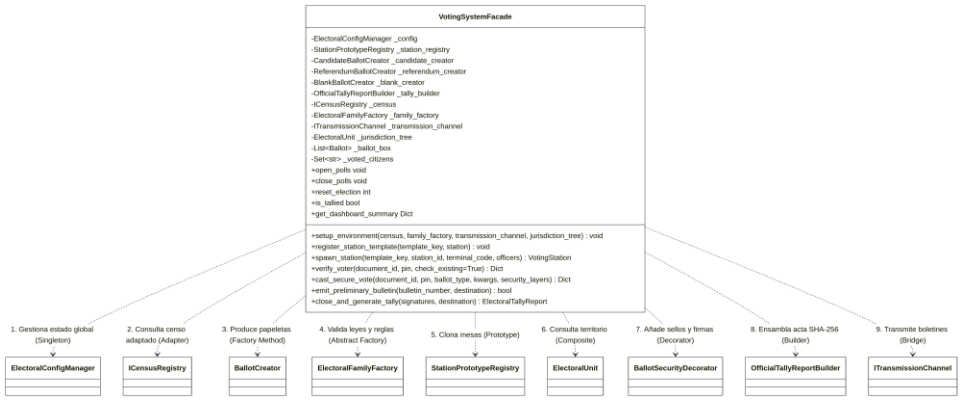
    def _compute_signature(self, signing_key: str, officer_token: str) -> str:
        canonical_content =
f"{self.ballot_id}:{self.get_selection()}:{self.issued_at.isoformat()}:{officer_token}"
        return hmac.new(
            key=signing_key.encode("utf-8"),
            msg=canonical_content.encode("utf-8"),
            digestmod=hashlib.sha256,
```

```
        ).hexdigest()

    def verify_signature(self, signing_key: str) -> bool:
        expected = self._compute_signature(signing_key, getattr(self,
"_officer_token"))
        return hmac.compare_digest(getattr(self, "_signature"), expected)
```

5. Parte C: Patrón de Diseño FACADE

5.1 Modelo Estructural y Diagrama de Clases (UML)



5.2 Implementación Técnica en Código (`src/modules/facade/voting_facade.py`)

La clase `VotingSystemFacade` concentra y coordina todos los subsistemas en métodos de alto nivel:

```
class VotingSystemFacade:
    """Fachada unificada de alto nivel para el sistema electoral."""

    def __init__(self, institution_name: str, election_title: str, election_type:
str = "NATIONAL") -> None:
        self.election_type = election_type.upper().strip()
        self._config = ElectoralConfigManager(institution_name, election_title)
        self._station_registry = StationPrototypeRegistry()
        self._candidate_creator = CandidateBallotCreator()
        self._referendum_creator = ReferendumBallotCreator()
        self._blank_creator = BlankBallotCreator()
        self._tally_builder = OfficialTallyReportBuilder()
        self._census: Optional[ICensusRegistry] = None
        self._family_factory: Optional[ElectoralFamilyFactory] = None
        self._transmission_channel: Optional[ITransmissionChannel] = None
        self._jurisdiction_tree: Optional[ElectoralUnit] = None
        self._tally_generated: bool = False

    def verify_voter(self, document_id: str, pin: str, check_existing: bool =
True) -> Dict[str, Any]:
        # Valida urna abierta, formato, PIN contra el censo y voto único
        ...

    def cast_secure_vote(self, document_id: str, pin: str, ballot_type: str,
                         selection_kwargs: Dict[str, Any],
                         security_layers: Optional[List[str]] = None) -> Dict[str,
Any]:
        # Valida elegibilidad, fabrica papeleta, aplica decoradores y persiste
        # la participación y la papeleta anónima en SQLite
        ...
```

```
    def close_and_generate_tally(self, officer_signatures: List[str],
                                 dispatch_destination: Optional[str] = None) ->
ElectoralTallyReport:
        # Cierra urnas, totaliza votos, construye acta consolidada con Builder y
transmite por Bridge
        ...
```

6. Análisis de Ingeniería de Software

6.1 Relación entre los patrones

El sistema articula los patrones estructurales para resolver los problemas de integración y coordinación:

1. **Adapter:** Resolvió la incompatibilidad del censo de electores (JSON vs CSV).
2. **Bridge:** Desacopló los boletines oficiales de los canales físicos de transporte (Consola, Archivo, Sink).
3. **Composite:** Jerarquizó el mapa electoral unificando el tratamiento de mesas individuales y recintos completos.
4. **Decorator:** Dotó a las papeletas de capas dinámicas de seguridad acumulativas (HMAC, marcas y tiempo).
5. **Facade:** Creó una interfaz unificada y limpia que encapsula toda la arquitectura.

6.2 Cumplimiento de Principios SOLID

- **Single Responsibility Principle (SRP):** El cliente web o API solo invoca a `VotingSystemFacade` para orquestar la jornada; la fachada delega cada subtarea a su respectivo módulo especializado.
- **Open/Closed Principle (OCP):** Nuevos decoradores de seguridad (ej. firma asimétrica Ed25519) pueden envolver a las papeletas sin modificar el código de la clase base `Ballot` ni de las fábricas.
- **Liskov Substitution Principle (LSP):** Cualquier `ElectoralUnit` (sea `VotingStationLeaf` o `ElectoralJurisdictionComposite`) puede ser consultada indistintamente para contabilizar mesas o recopilar identificadores.
- **Dependency Inversion Principle (DIP):** La fachada depende exclusivamente de abstracciones e interfaces (`ICensusRegistry`, `ITransmissionChannel`, `ElectoralFamilyFactory`, `ElectoralUnit`), inyectadas limpiamente mediante `setup_environment`.

7. Conclusión: Integración con la capa visual

Con la implementación de `VotingSystemFacade`, la arquitectura queda preparada para conectar la interfaz gráfica y los endpoints de FastAPI.

A partir de este punto, la capa web no requiere interactuar con fábricas ni constructores internos; cada controlador REST o formulario HTML interactúa directamente con los métodos limpios del Facade:

- `facade.verify_voter(documento, pin)`
- `facade.cast_secure_vote(documento, pin, tipo, seleccion)`
- `facade.get_dashboard_summary()`
- `facade.reset_election()`
- `facade.close_and_generate_tally(firmas)`

Patrones de Diseño Proxy y Flyweight

1. Identificación del patrón

- **Patrones de Diseño:** Proxy y Flyweight (Patrones Estructurales).
- **Elementos del Sistema:**
  - **Seguridad y Control de Acceso (Proxy):** `ElectoralAccessProxy`, modelo de sesión `UserSession`, y enumeración de roles `UserRole` (`ADMINISTRADOR`, `JURADO`, `AUDITOR`, `ELECTOR`).
  - **Optimización de Opciones Electorales (Flyweight):** `CandidateMetadataFlyweight` (Estado Intrínseco) y pool centralizado `CandidateFlyweightFactory`.
- **Ubicación del Código:**
  - `src/modules/security/access_proxy.py`
  - `src/modules/voting/candidate_flyweight.py`

2. Justificación y Problema que Resuelven en el Dominio

El sistema aborda dos retos concretos de la plataforma: **la seguridad de privilegios** y **el catálogo compartido de candidatos**.

2.1 Justificación de PROXY: Control de Privilegios y Auditoría de Operaciones

En un sistema electoral, no todos los usuarios pueden ejecutar cualquier operación:

- Solo los **ADMINISTRADORES** pueden abrir formalmente la jornada electoral o reconfigurar los parámetros de las mesas.
- Solo los **JURADOS DE MESA** o electores autenticados pueden registrar votos en la cabina.
- Solo los **JURADOS y ADMINISTRADORES** debidamente identificados pueden cerrar urnas y firmar el Acta de Escrutinio.
- El rol **AUDITOR** está definido para fiscalizar resultados y estado de urnas; en la demo la consulta pública del dashboard no exige sesión.

El Problema de Seguridad sin Proxy: Si expusiéramos directamente el `VotingSystemFacade` al cliente web o API, tendríamos que ensuciar la lógica de negocio con verificaciones de permisos repetitivas en cada método, violando el Principio de Responsabilidad Única (SRP).

Solución Proxy: `ElectoralAccessProxy` envuelve la fachada y actúa como un sustituto o intermediario protector. Intercepta cada solicitud antes de que llegue al núcleo:

1. Verifica si el rol de la sesión cuenta con privilegios legales para la acción solicitada.
2. Registra un evento de auditoría en la bitácora (`_audit_log`), documentando quién, cuándo y con qué resultado se intentó la operación.
3. Si la verificación es exitosa, delega la llamada a `VotingSystemFacade`; si no, bloquea la acción arrojando `PermissionError`.

2.2 Justificación de FLYWEIGHT: Catálogo Compartido de Candidatos

Cada sufragio emitido por un candidato contiene información descriptiva que no cambia entre votos:

- Nombre completo del candidato.
- Partido político o movimiento ciudadano.
- Lema de campaña.
- Logotipo y color oficial del tarjetón.

El Problema de Duplicación: Si cada papeleta instanciara sus propias copias de estos textos, se repetiría

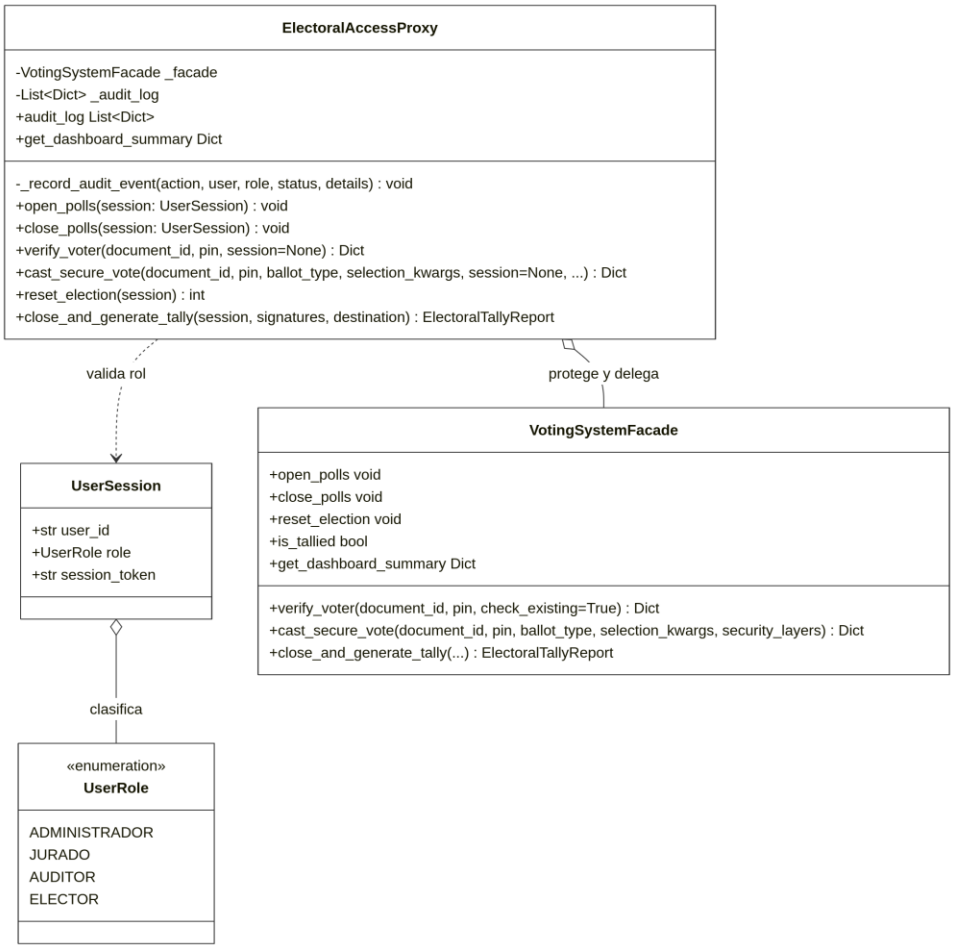
información innecesariamente. En esta demo el beneficio es de claridad y consistencia; la escala real sería mayor en una elección masiva.

Solución Flyweight (Separación de Estados):

- **Estado Intrínseco (Compartido e Inmutable):** Modelado en `CandidateMetadataFlyweight`. Contiene los datos fijos del candidato (`candidate_id`, `candidate_name`, `party_name`, `slogan`, `color`). Este objeto existe **una única vez en memoria por candidato** y se comparte en el catálogo y en la validación server-side. La papeleta persistida guarda el `candidate_id`, no una copia completa de la ficha.
- **Estado Extrínseco (Único por voto):** Modelado en la papeleta individual (`ballot_id`, `issued_at`, `receipt_code`).
- **Criterio contra la sobreingeniería ("Patternitis"):** En lugar de crear jerarquías pesadas innecesarias, se implementó `CandidateFlyweightFactory` de forma compacta y legible (~35 líneas), proporcionando además el catálogo oficial para renderizar el tarjetón visual en la interfaz web.

3. Parte A: Patrón de Diseño PROXY

3.1 Modelo Estructural y Diagrama de Clases (UML)



3.2 Implementación Técnica en Código (`src/modules/security/access_proxy.py`)

```
class ElectoralAccessProxy:
    """Proxy de protección y auditoría sobre VotingSystemFacade."""

    def __init__(self, target_facade: VotingSystemFacade) -> None:
        self._facade = target_facade
```

```
        self._audit_log: List[Dict[str, Any]] = []

    def open_polls(self, session: UserSession) -> None:
        """Permite abrir la urna únicamente a usuarios con rol ADMINISTRADOR."""
        if session.role != UserRole.ADMINISTRADOR:
            self._record_audit_event("OPEN_POLLS", session.user_id,
session.role.value, "RECHAZADO")
            raise PermissionError("Acceso denegado: Se requiere rol ADMINISTRADOR
para abrir la jornada electoral.")

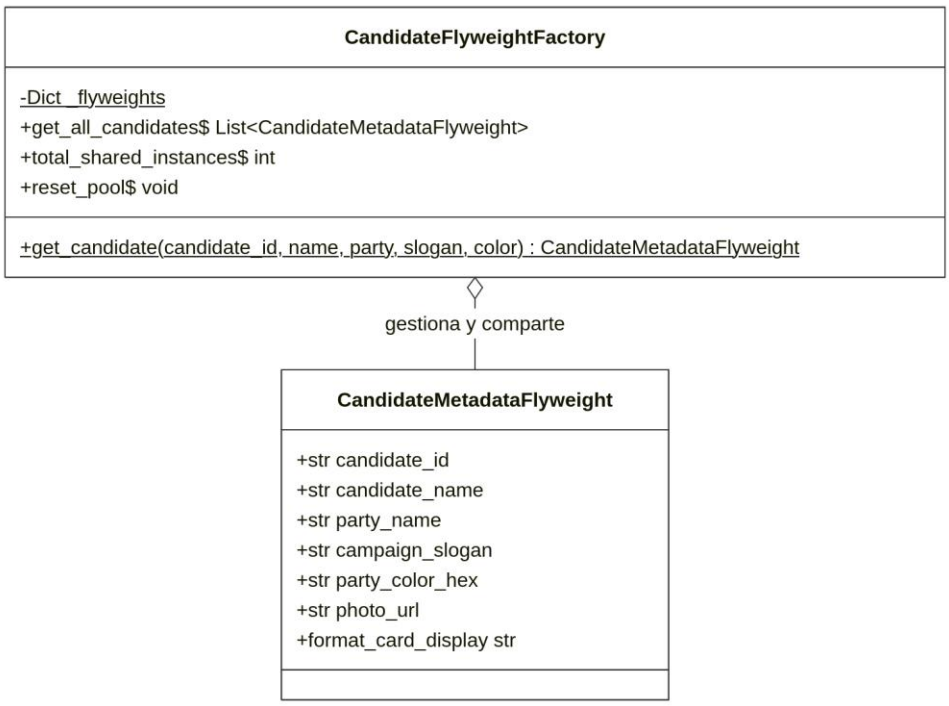
        self._record_audit_event("OPEN_POLLS", session.user_id,
session.role.value, "AUTORIZADO")
        self._facade.open_polls()

    def close_and_generate_tally(
        self,
        session: UserSession,
        officer_signatures: List[str],
        dispatch_destination: Optional[str] = None,
    ) -> ElectoralTallyReport:
        """Controla el escrutinio final exigiendo firmas de jurados y
privilegios."""
        if session.role not in (UserRole.ADMINISTRADOR, UserRole.JURADO):
            self._record_audit_event("GENERATE_TALLY", session.user_id,
session.role.value, "RECHAZADO")
            raise PermissionError("Acceso denegado: Solo jurados o administradores
pueden consolidar el acta final.")

        report = self._facade.close_and_generate_tally(
            officer_signatures=officer_signatures,
            dispatch_destination=dispatch_destination,
        )
        self._record_audit_event("GENERATE_TALLY", session.user_id,
session.role.value, "CONCLUIDO")
        return report
```

4. Parte B: Patrón de Diseño FLYWEIGHT

4.1 Modelo Estructural y Diagrama de Clases (UML)



4.2 Implementación Técnica en Código (`src/modules/voting/candidate_flyweight.py`)

```
@dataclass(frozen=True)
class CandidateMetadataFlyweight:
    """Estado intrínseco e inmutable compartido en el catálogo de candidatos."""
    candidate_id: str
    candidate_name: str
    party_name: str
    campaign_slogan: str
    party_color_hex: str
    photo_url: str

class CandidateFlyweightFactory:
    """Fábrica y almacén común (Pool) para compartir instancias únicas en
memoria."""

    _flyweights: Dict[str, CandidateMetadataFlyweight] = {}

    @classmethod
    def get_candidate(cls, candidate_id: str, **kwargs) ->
CandidateMetadataFlyweight:
        c_id = str(candidate_id).strip()
        if c_id not in cls._flyweights:
            cls._flyweights[c_id] = CandidateMetadataFlyweight(candidate_id=c_id,
**kwargs)
        return cls._flyweights[c_id]
```

5. Análisis de Ingeniería de Software

5.1 Cumplimiento de Principios SOLID

- **Single Responsibility Principle (SRP):**

    - `VotingSystemFacade` solo se encarga de la lógica orquestada del dominio electoral.
    - `ElectoralAccessProxy` asume la responsabilidad de validar el rol de la sesión y registrar la auditoría; la emisión de la cookie la realiza `src/presentation/auth.py`.
    - `CandidateFlyweightFactory` se ocupa exclusivamente de mantener el catálogo compartido de candidatos.
- **Open/Closed Principle (OCP):** Es posible incorporar nuevos tipos de proxies (ej. `CachingProxy` para respuestas rápidas de dashboard) o nuevos roles de acceso sin modificar el código interno del Facade.
- **Liskov Substitution Principle (LSP):** El cliente interactúa con el Proxy utilizando las mismas operaciones esperadas del sistema electoral.

5.2 Balance arquitectónico

Con la incorporación de **Proxy** y **Flyweight**, el catálogo de patrones estructurales queda implementado con una justificación de dominio sólida:

#	Patrón Estructural	Módulo	Problema que Resuelve en el Dominio
1	**Adapter**	`src/modules/census/`	Homogeneiza censos externos heterogéneos (JSON vs CSV).
2	**Bridge**	`src/modules/transmission/`	Desacopla boletines de salida de los canales físicos de transporte.
3	**Composite**	`src/modules/election/`	Jerarquiza el mapa electoral y mesas en estructura parte-todo.
4	**Decorator**	`src/modules/voting/`	Añade capas dinámicas de seguridad al voto sin explosión de clases.
5	**Facade**	`src/modules/facade/`	Unifica todos los subsistemas en una interfaz limpia de alto nivel.
6	**Proxy**	`src/modules/security/`	Protege operaciones sensibles con validación de roles y auditoría.
7	**Flyweight**	`src/modules/voting/`	Comparte los metadatos de los candidatos y evita copiar la ficha completa en cada papeleta.